

第2回

手嶋毅志 (Takeshi Teshima)



質問と回答

加算対象ではないその他の質問・コメント

AIには「予測系タスク」や「生成系タスク」など、目的によって様々な種類があるというお話があり、非常に興味を持ちました。

ありがとうございます！その中でも「予測系タスク」は他のタスクの技術的な土台となることも多く、基本的な考え方の多くが現れるので、この講義ではそこから入っていきます。

もし先生の中で「このタスクにはこのAIが向いている」という使い分けの基準や、個人的に「これは性能が高いのでおすすめだ」と考えているツールがあれば、ぜひ教えていただきたいです。

基準はありませんが、個人的には次のような使い分けをしています。ただし、来月には全然違うものを使っているかもしれないぐらいなので、すごく強いこだわりはありません。（続く）

質問（続き）

（私のツールの使い分けは？の続き）

- 普段使い：Gemini（ブラウザやアプリから利用。Google AI Proプランの課金です）
 - 数学などの専門性の高い内容はGemini 3.1 Pro，簡単な質問などはGemini 3 Flash，多くの情報を集めたいときはDeep Researchモードで使う。
- プログラミング：①エディタはAntigravityかGitHub CopilotかCursorで，中身はGeminiやClaude等。②コーディングエージェントをCLIで呼ぶ場合，最近Gemini CLIを使っています。
- 特に難易度の高いことをする場合：Claudeに単月で課金してClaude CoWorkを使うかも
- 他のTIPS：①ハルシネーション（＝AIによる悪意のない情報の捏造）には気をつけよう，②学習のopt-out可かどうかは必ず確認しよう，③評判が良いものがあればどんどん試して乗り換えよう（例：Geminiが画像データの扱いで優っていた時期があるが，今では大差ない）

加算対象ではないその他の質問・コメント

過去のデータから予測器を手に入れると思うのですが、その予測器が良いか悪いかは、これから現れる未知のデータを入力してみない限りはわからないのですか。

はい、予測器が良いか悪いが、本当の意味では、これから現れる未知のデータを入力して結果をみるまで分かりません。それどころか、応用場面によっては、良い予測だったかどうか永遠に不明なままということもあり得ます。

今回、まさしくその話しますのでお楽しみに。

加対象の質問

最近ChatGPTやGeminiなど多くのAIが登場していますが、これらはやはりツールごとに「予測が得意」「生成に特化している」といった明確な強みの違いがあるのでしょうか？

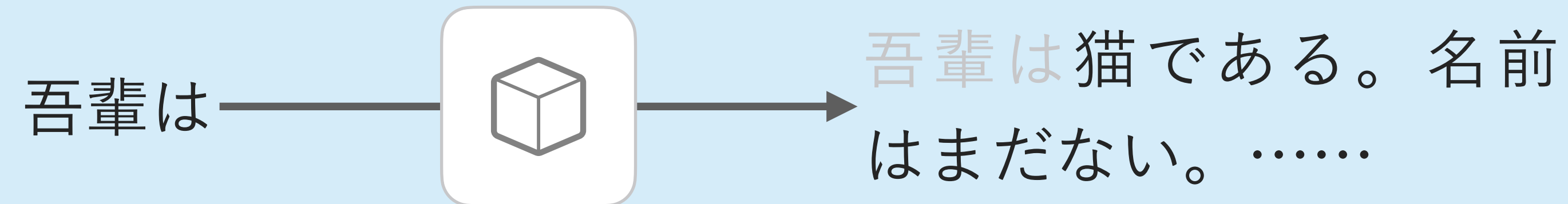
実は、ChatGPTやGeminiは、どちらも「生成タスクのためのAI」であり、それ以上でもそれ以下でもありません（！）これらは「生成AI」というバズワードで総称されることがありますが、それは生成タスクを解くためのAIだからです。

ではこれを、予測タスクなどのために使えるかというと……？

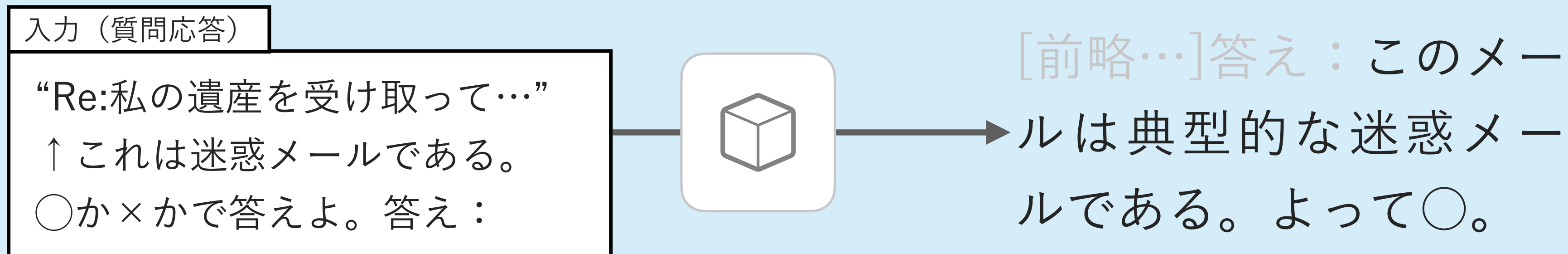
理解のためには、まずは基本はNOだと思っておいたほうがよいですが、
一歩進んだ話をしてしまうと、現在は「Yes and No」です。（続く）

質問（続き）

GPTやGeminiといったモデルは、基本は**文章の続き**を書ける**生成器**です。



ところが、超高品質な生成ができるようになってきた結果、驚くべきことに、それが他のタスクを「擬似的に」解くのに使える場合が出てきました。



質問（続き）

とはいえ，これは**高性能な生成器**の「意外な使い道」ぐらいの話。

予測タスクを，本来は生成のための技術を使って，何でもかんでも解こうとしてしまうとミスマッチです。

（とは言っても，例えば文書分類のタスクなどでは実用レベルで使えるので，ものは使いようですね）

本格的に腰を据えて予測タスクを解く際には，そのための手法でそのためのモデルを学習したほうが性能としては高くなるのが普通です。

これらの話は講義の最後の数回でも詳しく出てくるので，それまで一旦忘れておいてください。

加対象の質問

出現確率の低い入力は多少おろそかにしても影響は軽微とありますが、どのくらいの誤差なら許容できるのですか。

良い質問ですね！

つまらない回答としては「ケースバイケース」ということになりますが……

重要な考え方として「ベースラインを確立する」という話を説明します。（つづく）

質問（続き）

実用の場面でどの程度の性能が出れば十分と見做されるかはケースバイケースです。

例えば、医療の診断で「平均すると70%ぐらい当たります」と言われたら、不安だ……と思いますが、不動産の購入者予測で「購入するかどうか60%ぐらいで当てられます」と言われたら、1件の購入単価が高いので、そのモデルを使って営業をかける価値はあるかもしれません。外れた場合の実害の大きさによって、許容不可能な予測誤差の水準は変わります。

また、分野ごとに「このぐらいは達成できるはず」という予測誤差の水準も、証拠はないですが、多少の暗黙知がある場合もあります。この分野のこのタスクならこの損失の期待値は0.1未満までいけるはず、というような。

質問（続き）

そうは言っても「やってみるまでどれぐらいの性能が出そうか全く分からない」という場面も多いです。そこで重要になるのが「ベースラインを確立する」という考え方です。

ベースラインとは、「基準となる予測手法や、そのときの予測性能（リスク）」のこと。重要なベースラインとしては「単純集計モデル」や「人間の精度」があります。

単純集計によるモデル

特徴量をいくつかの値で区切って、その中でのラベルの分布を見ます。例えば「 x_1 」の値を10ずつに区分けする。そして、その区分けでの訓練データのラベルの分布を見て、最も頻度が高かったラベルを予測ラベルとする（回帰なら、その区分での平均を予測値とする）」など。予測器としては（おそらく最も）単純ですが、だからこそ「最低限達成できてほしい性能」を考える助けになります。

質問（続き）

人間の精度

人間の専門家の精度をベースラインとする場合もあります。例：医療での画像診断
これをベースラインとするということはつまり、人間の医師による予測の誤分類率を基準として考える、ということです。

例えば人間の医師の誤分類率が1%未満のタスクで、学習したモデルが2%の誤分類率なら、まだまだ改善の余地があることが分かりますね。

元の質問に戻ると「どのくらいの誤差なら許容できるのですか。」でしたが、実用上は逆の考え方をして「これぐらいは達成できていなければ性能が足りない」と判断できる基準を用意する（基準を確立する方法のアイデアを出す）と考えると、一歩前進できることが多いように思います。

加対象の質問

第一次と第二次のAIブームにおいてはいずれも冬の時代を迎えたのですが、冬の時代を迎えなかった第三次ブームで、今後何らかの深刻な欠陥等により冬の時代に入ってしまう、ということは考えられますか？

良い質問ですね！現在は第三次を終えることなくそのまま第四次ブームとさえ言われますが、**今回のブームで生まれた技術が廃れることは、まず無い**と思います。今よりさらに普及が進み、「変化」としてのブームは終わってゆくかもしれませんが、**重要性は減ることはなく、むしろ増すばかり**です。

ただし、進歩の停滞に繋がりうる要素がいくつか存在するので、ブームとしてはいずれ終わりを迎える（冬の時代ではなく、成功裡に終息）かもしれません。（つづく）

質問（続き）

1. 学習データの枯渇（2026年問題）

2026年頃にインターネット上の「学習に値する高品質テキストデータ」（書籍や論文や記事など）は学習に全て使われた状態になってしまうとする向きもある

新たな学習データが生まれる速度が足りなくなるということ。

（品質やデータの種類をもっと広く取ると、2050年頃まで持つともされる）

2. 経済的・環境的な限界（エネルギーとコスト）

莫大な投資が行われているので、見合う効果が出なければ投資引き上げになる懸念
投資は先読みで行われ、期待を上回り続けている現状ですが、いつまでも続くか？

3. 信頼性と法規制の壁

性能が上がりすぎると法規制の対象になる。

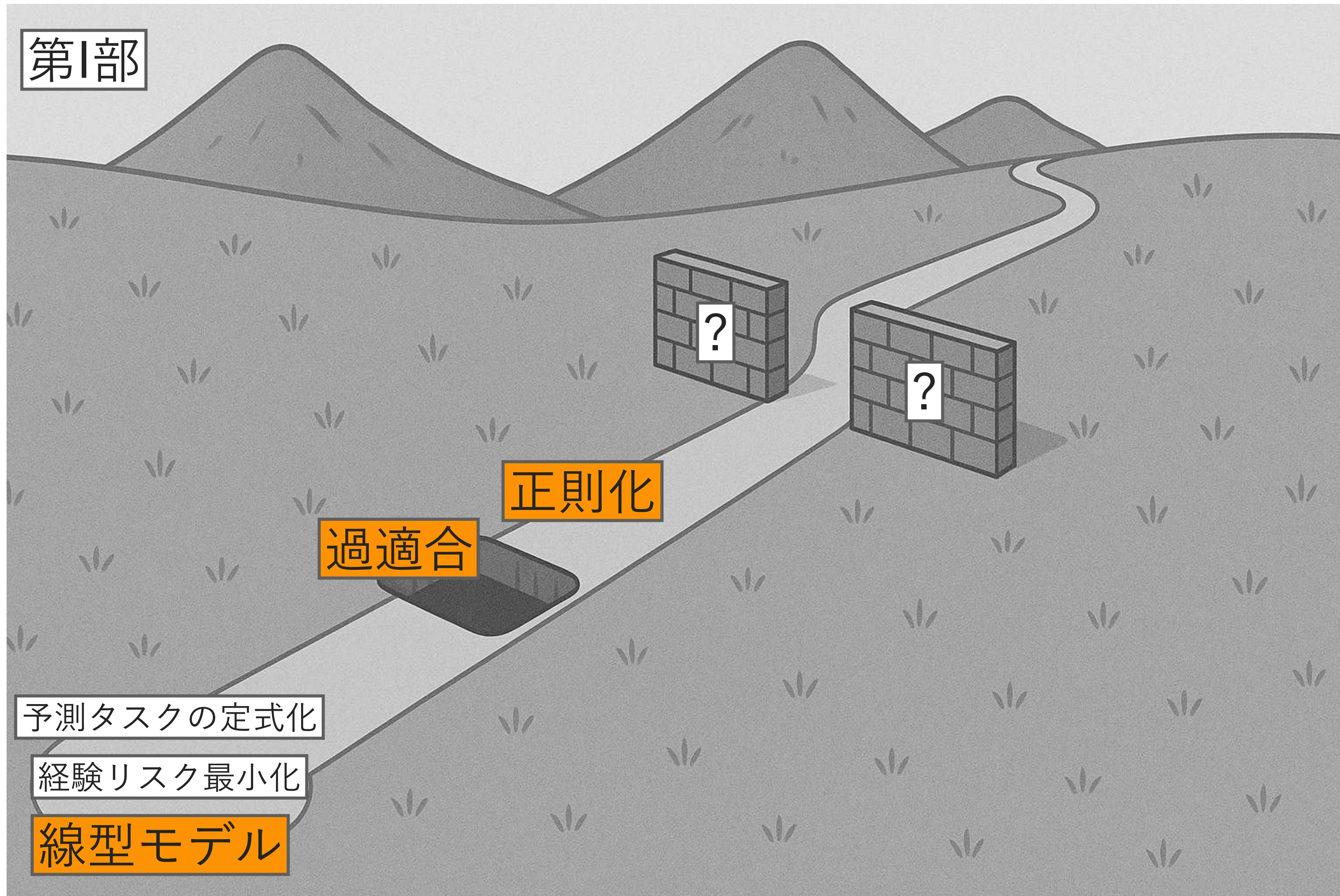
過去の冬の時代が「何もできなかった」ことによる冬だとしたら今回は起きない。

「これ以上（統計的・経済的・物理的に）先に進めない」即ち「飽和による停滞」か。

告知

- 第1回の小テスト，公開済みです
- 第2回の小テストも講義後に公開します

ロードマップ



(前回の続き)

予測系タスクの学習 > 基本のアイデア

「教師付き学習」の基本のキーアイデアを3ステップで説明

- キーアイデア① **?** 良い関数をどうにか見つける **○** モデルのパラメーターを調整
 - 様々な関数形を人が頑張って考えるのではなく、**モデル**（ここではパラメーター付き関数のこと）を用意して、**パラメーター**の調整で色々な関数を作る
- キーアイデア② **?** あらゆる入力で正確な予測 **○** 平均的には正確な予測
 - 全てのデータ点で正しい予測（＝予測誤差が0）を目指すことは一旦諦めて、代わりに、予測誤差の**期待値**がなるべく小さいパラメーターを探す
- キーアイデア③ **?** 誤差の期待値を小さく **○** 訓練データでの誤差の平均を小さく
 - **予測誤差の期待値**は計算不可能なので、代わりに**標本平均**を使う（データはそのため*i.i.d.*サンプルだと仮定する）

これまでのあらすじ

- 予測器を作りたい（特徴量を代入するとラベルの予測値が出てくる関数）



- モデル f_θ を選び、損失関数 ℓ を決め、期待損失が小さいパラメーター θ を探すことにした

$$R(f_\theta) := \mathbb{E} [\ell(Y, f_\theta(X))] \leftarrow \text{リスク関数}$$

- ありうる全ての入出力で正解するのは至難の業なので、平均的な良さを目指す
- （前回の続きの話）実は、リスクは、概念としては定義できても、実際には計算できないという問題が潜んでいる

予測系タスクの学習 > ゴール設定の落とし穴



リスク関数を最小化すればいいのね！

- 落とし穴：データ (X, Y) の生成分布 $p(x, y)$ は未知

$$R(f) := \int \ell(y, f(x)) \underset{\substack{\uparrow \\ \text{未知}}}{p(x, y)} dx dy$$

- よって、 $R(f_\theta)$ が小さくなるようにパラメーターを探すどころか、1つの予測器 f_θ に対する $R(f_\theta)$ の値を求めることすらできない
- 「 $R(f_\theta)$ が小さくなるようにパラメーター θ を調節」はあくまで仮想的な目標であり、そのまま実現はできない

予測系タスクの学習 > ゴール設定の落とし穴の回避



ではどうすればよいか？ 期待値を，標本平均で近似しよう！

- 手持ちのデータが一定の条件を満たすならば，データ量が増えれば標本平均は期待値の良い近似であることが期待できる（cf. 大数の法則や中心極限定理）

$$\frac{1}{n} \sum_{i=1}^n X_i \quad \rightsquigarrow \quad \mathbb{E}[X]$$

標本平均 期待値

- そこで，手持ちのデータを使って， $R(f)$ を標本近似することにする．

予測系タスクの学習＞データに関する仮定

データに関する仮定

- **仮定**：訓練データは、テストデータと同じ分布から取ったIIDサンプル

$$\{(X_i, Y_i)\}_{i=1}^n \stackrel{\text{i.i.d.}}{\sim} p$$

- ここで、 p はデータの生成分布（手持ちのデータとテストデータで共通）
-
- この仮定は何を意味するか？
 - 手持ちのデータで、リスク関数を近似することを正当化するための仮定

予測系タスクの学習 > リスクの標本近似

リスク $R(f)$ を手持ちの標本で近似する

■ リスク

$$R(f) := \mathbb{E} [\ell(Y, f(X))]$$

……期待値
(近似したい対象)

■ 経験リスク (empirical risk)

$$\hat{R}(f) := \frac{1}{n} \sum_{i=1}^n \ell(y_i, f(x_i))$$

……標本平均

【補足】 区別のために、リスクのことを期待リスクや母集団リスク (population risk) ということも稀にある (正式には単にリスクと呼ぶ)。特に断らなければ「期待」と言わなくてもリスクは損失の期待値のことを指す。

予測系タスクの学習 > 経験リスク最小化

経験リスクならデータから計算できるので、これをリスクの代わりに最小化

■ 経験リスク最小化 (empirical risk minimization, ERM)

- 経験リスクが最小になるようにパラメーターを決めるという手法

$$\hat{R}(f_{\theta}) := \frac{1}{n} \sum_{i=1}^n \ell(y_i, f_{\theta}(x_i)) \rightarrow \text{最小化}$$

- これは「教師付き学習」の基本的な型の一つ

これまでのあらすじ

- 予測器を作りたい（特徴量を代入するとラベルの予測値が出てくる関数）



- モデル f_{θ} を選び、損失関数 ℓ を決め、期待損失が小さいパラメーター θ を探す

$$R(f_{\theta}) := \mathbb{E} [\ell(Y, f_{\theta}(X))] \leftarrow \text{計算不可能だが小さくしたい}$$

- 現実的には、損失のサンプル平均の最小化なら実行できることを紹介

$$L(\theta) := \hat{R}(f_{\theta}) := \frac{1}{n} \sum_{i=1}^n \ell(y_i, f_{\theta}(x_i)) \rightarrow \text{Minimize}$$

- 具体例は次のセクション

経験リスク最小化に基づく学習

線型単回帰の場合

入力が1次元であるような回帰（単回帰；simple regression）

- ここでは、モデルは1次多項式だけを考えることにする

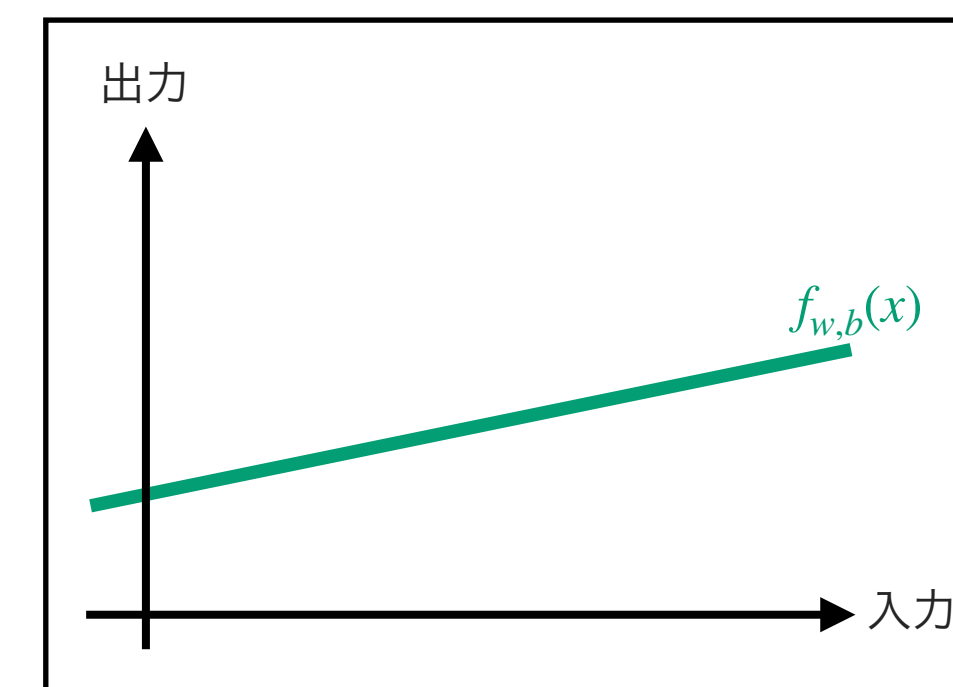
$$f_{w,b}(x) = wx + b \quad (w, b \in \mathbb{R})$$

- 損失関数は二乗損失とする

$$\ell(y, y') = (y - y')^2$$

- この場合のリスクは期待二乗誤差 → この値が小さい (w, b) を求めたい

$$R(f_{w,b}) := \mathbb{E} [\ell(Y, f_{w,b}(X))] \rightarrow \text{計算できない}$$



【補足】 モデルが1次関数の場合なので、これは線型単回帰（simple linear regression）という。

【用語】 この b は数学では切片（intercept）と呼ばれるが、機械学習の用語では「バイアス」（bias）と呼ばれることも多い。

線型単回帰の場合＞データの記号と仮定

訓練用のデータセットは持っているとする

- 訓練データはラベル付きデータセット．次の記号で表すことにする．

$$\{(x_i, y_i)\}_{i=1}^n$$

- 今回は，単回帰なので， x も y も単なる数値 ($x_i \in \mathbb{R}$, $y_i \in \mathbb{R}$) ．

X	Y
0.20	0.53
12.3	36.0
-2.7	-7.29

イメージ

- これは，予測タスクを実際に行う際の未知データと同じ生成分布からの i.i.d. サンプルだと仮定しておく

$$\{(X_i, Y_i)\}_{i=1}^n \stackrel{\text{i.i.d.}}{\sim} p_{X,Y}$$

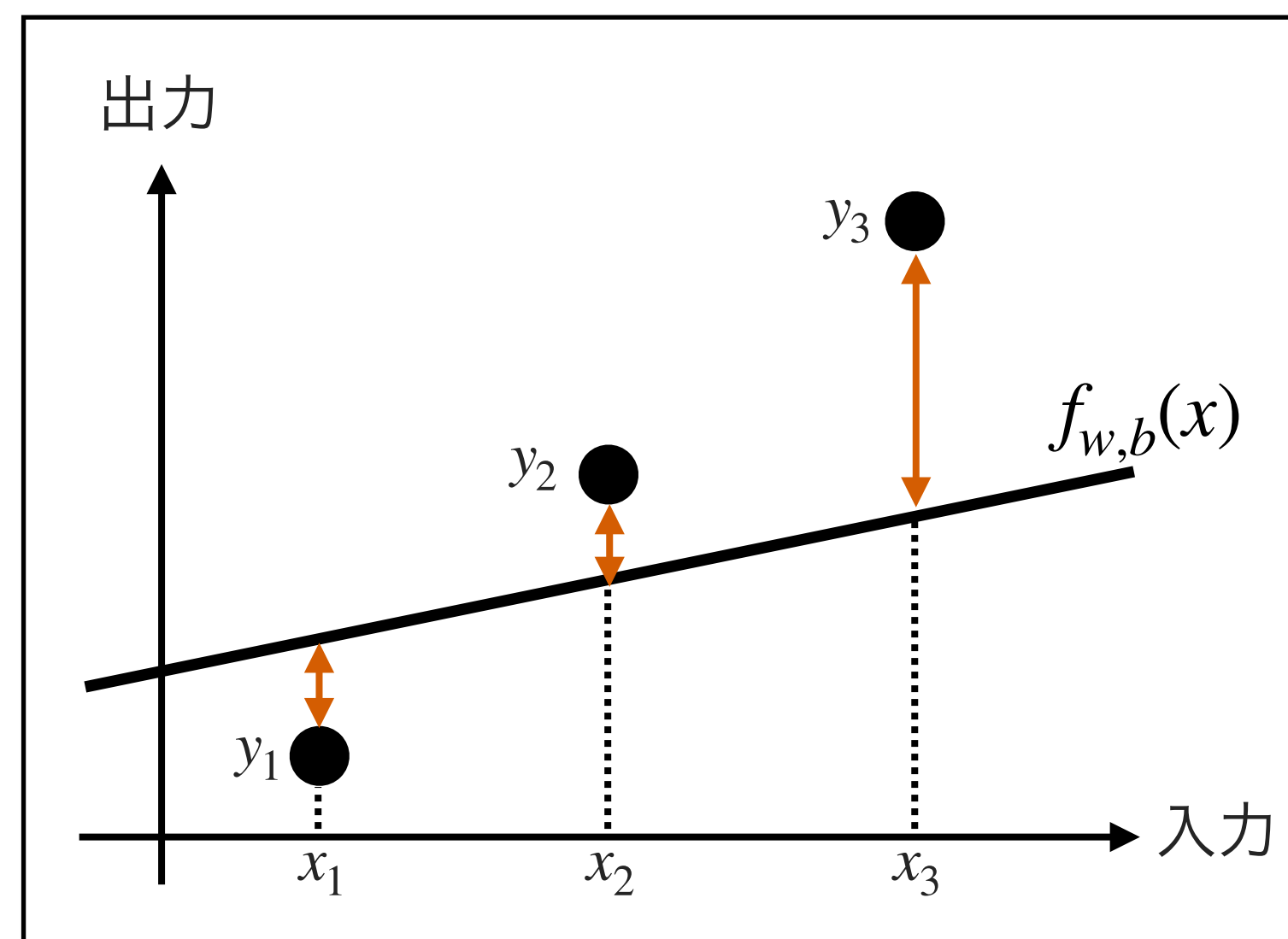
線型単回帰の場合＞経験リスク

リスクそのものは見えないので、代わりに経験リスクの最小化で学習をする

- この場合の経験リスクは、**平均二乗誤差 (mean squared error, MSE)**

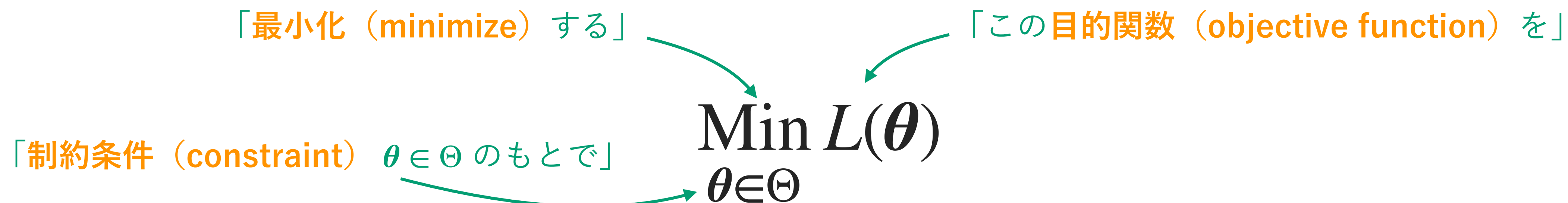
$$L(w, b) = \frac{1}{n} \sum_{i=1}^n (y_i - f_{w,b}(x_i))^2$$

(真値 - 予測値)²



基本知識の確認＞最適化問題

目的に対して最適な選択肢を求める問題（ここでは最小化問題だけ説明）



- ゴールは、「 $L(\theta)$ が最小になる入力 $\theta \in \Theta$ 」を（少なくとも一つ）見つけること
 - そのような θ^* を**最小化元 (minimizer)** や**最適解 (optimal solution)** という
- 読みやすさに応じて、次の表記も用いる：「 $L(\theta) \rightarrow$ 最小化 ($\theta \in \Theta$)」

【補足】 L は実数値関数。最大化問題は $-f(x)$ を最小化すれば等価なので、基本的に最小化問題だけで説明する。【用語】 最適解により達成される目的関数値 $L(\theta^*)$ を最適値という。【記号】 説明不要で通じやすい記号を採用したつもりだが、本講義の外では通じるとは限らないため注意が必要。数学では $\min\{L(\theta) : \theta \in \Theta\}$ で最適値を表すが、最適化問題そのものを表す記号は定まっておらず文献によって異なる。ここでは大文字の M を用いることで区別している。

線型単回帰の場合＞経験リスク最小化

今回の経験リスク最小化は，具体的には……

- 最小二乗法（ordinary least squares）による学習

$$\text{Min}_{(w,b) \in \mathbb{R}^2} \frac{1}{n} \sum_{i=1}^n (y_i - wx_i - b)^2$$

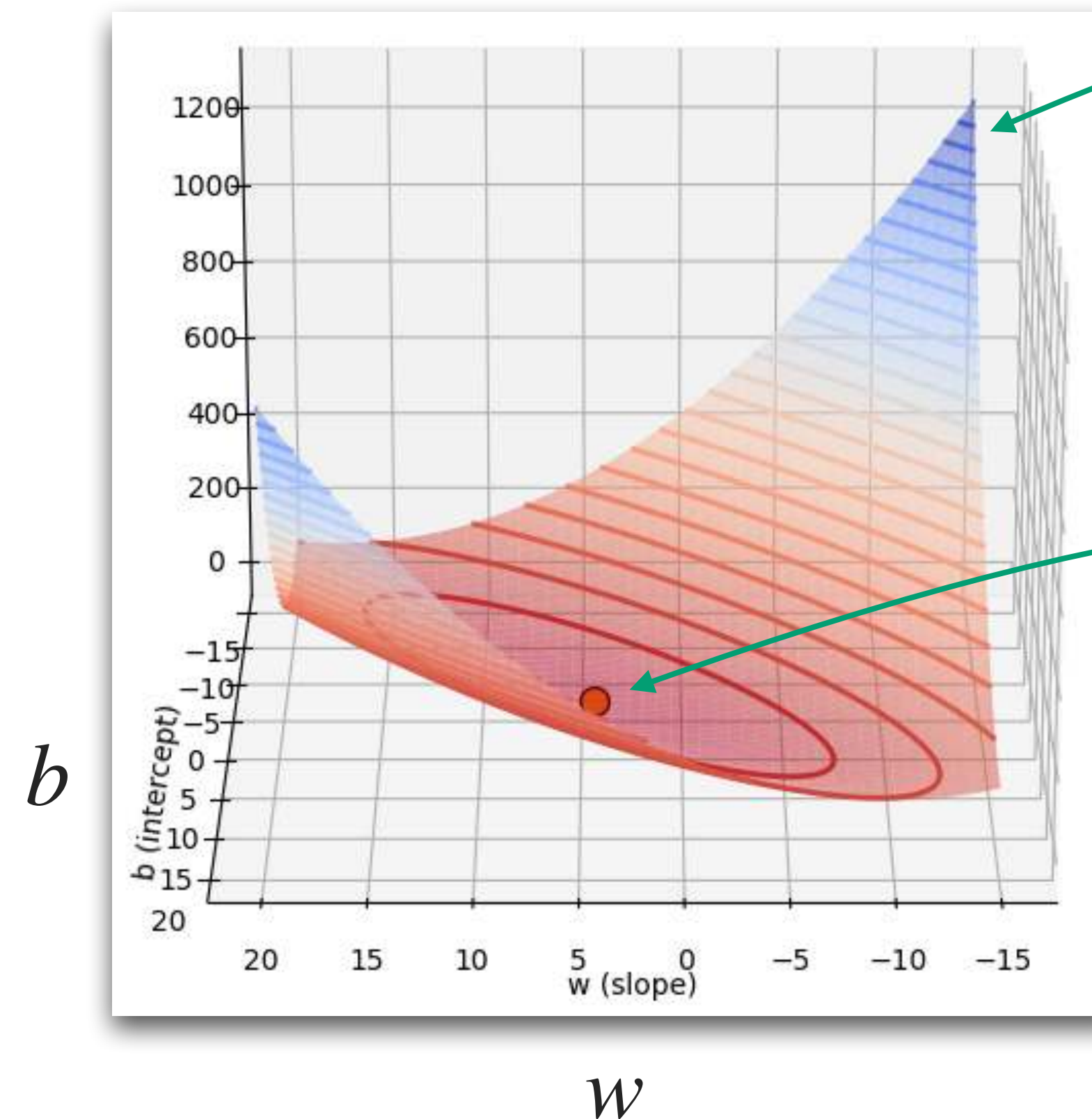
- 「平均二乗誤差を目的関数とする，制約なし最小化問題」

- この場合の学習とは，上記を解いて最適解 $(\hat{w}, \hat{b})$ を求めることにほかならない

線型単回帰の場合＞経験リスク最小化

視覚的には「次の目的関数を最小化する点 (w, b) を求めよ」

$$L(w, b) = \frac{1}{n} \sum_{i=1}^n (y_i - wx_i - b)^2$$



2変数の関数なので
曲面としてプロットできる

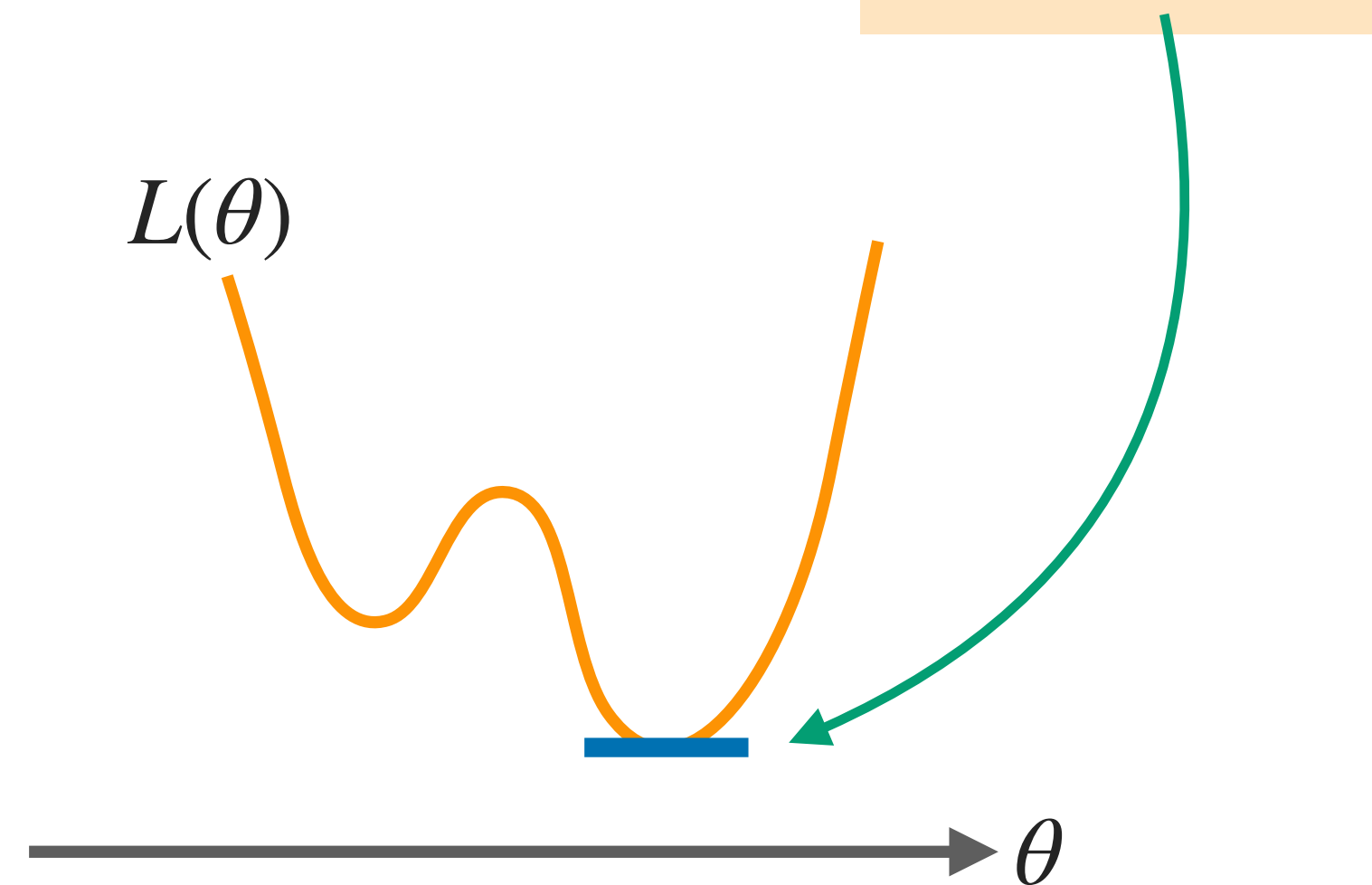
この曲面の底にあたる
 $(\hat{w}, \hat{b})$ が最適解

基本知識の確認＞最適化の一階の条件

最小化問題を解くには？（1変数の場合の復習）

- よく使われるのは「最適化の**一階の条件**（first-order condition）」
 - 制約なし最小化の場合、最適解は「微分して0」を満たす（必要条件）

$\text{Min}_{\theta \in \mathbb{R}} L(\theta)$ の最適解 θ^* において、 $\frac{dL}{d\theta}(\theta^*) = 0$ が成立



【注意】説明のために省略しているが、正確には、 L が連続微分可能であるなど、適当な仮定を満たす場合にのみ上記が成立する。

【補足】制約付き最適化の場合は、一階の条件として「微分して0」の代わりに、KKT条件（Karush–Kuhn–Tucker conditions）というものがある。

基本知識の確認＞偏微分

多変数関数の最適化では、偏微分が必要になる

■ 偏微分とは

多変数関数を特定の1変数だけで微分する操作

$$f(x, y) = x^2 + xy - y^2 \text{ を } x \text{ で } \longrightarrow 2x + y$$

y は本来は変数だが
 x で偏微分するときは定数とみなして扱う

■ 記号

- 偏微分した結果（偏導関数）は $\frac{\partial f}{\partial x}$, $\frac{\partial f}{\partial y}$ や, $\frac{\partial}{\partial x} f$, $\frac{\partial}{\partial y} f$ など表す.

基本知識の確認＞最適化の一階の条件

最小化問題を解くには？（多変数の場合）

- 多変数の場合の一階の条件は、勾配というものを使って表されることになる
- **勾配（gradient）**：偏微分を並べた縦ベクトルのこと

$$\nabla L(\boldsymbol{\theta}) = \begin{pmatrix} \frac{\partial}{\partial \theta_1} L(\boldsymbol{\theta}) \\ \vdots \\ \frac{\partial}{\partial \theta_r} L(\boldsymbol{\theta}) \end{pmatrix}$$

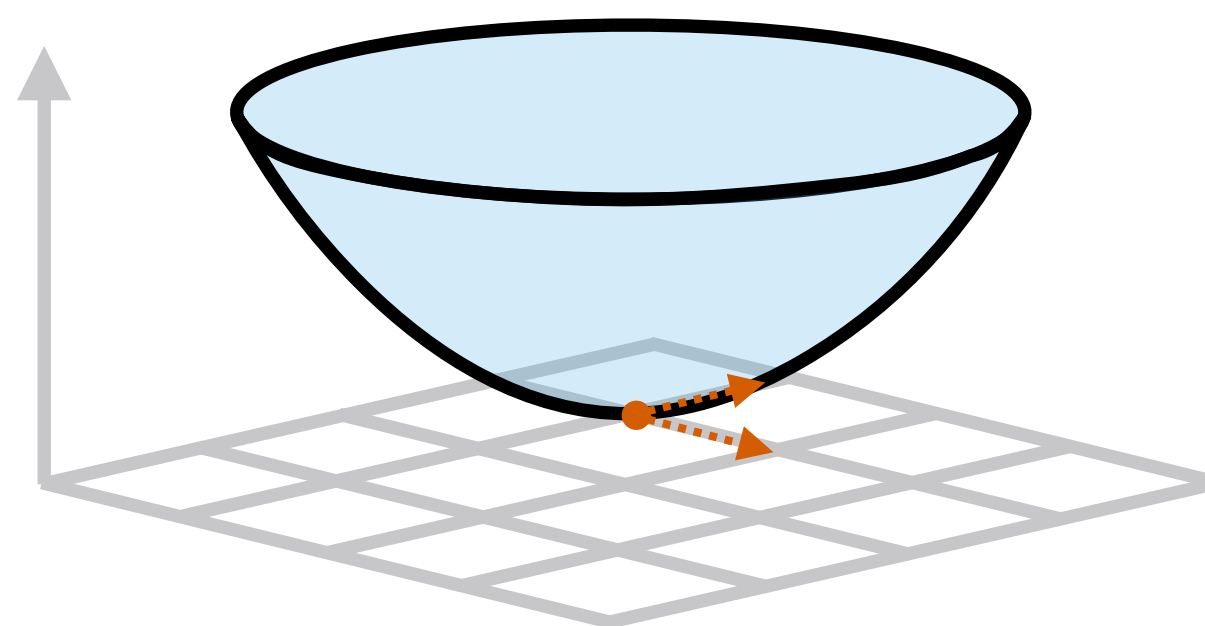
基本知識の確認＞最適化の一階の条件

最小化問題を解くには？（多変数の場合）

■ 最適化の一階の条件（多変数の場合）

- 制約なし最小化の場合，最適解は「勾配をとると0」を満たす（必要条件）

$\text{Min}_{\theta \in \mathbb{R}^d} L(\theta)$ の最適解 θ^* において， $\nabla L(\theta^*) = \mathbf{0}$ が成立



- ▶ 各変数について同時に「偏微分して0」が成立する，という条件

【注意】説明のために省略しているが，正確には，が連続微分可能であるなど，適当な仮定を満たす場合にのみ上記が成立する。

【補足】一階の条件が「勾配＝0」となるのは，制約なし最適化だからである．制約付き最適化の場合は，代わりにKKT条件（Karush–Kuhn–Tucker conditions）と呼ばれる条件が一階の条件になる．もちろん，制約なし最適化の場合の「勾配＝0」という条件は，KKT条件の特殊ケースである．

線型単回帰の場合 > 最小二乗法 > 最適化手法

実際に最適なパラメーターを求めてみよう

- 目的関数（経験リスク）

$$L(w, b) := \frac{1}{2} \sum_{i=1}^n (y_i - (wx_i + b))^2$$

- 勾配を定義から計算しよう

$$\nabla L(w, b) := \begin{pmatrix} \frac{\partial}{\partial w} L(w, b) \\ \frac{\partial}{\partial b} L(w, b) \end{pmatrix} = \begin{pmatrix} ? \\ ? \end{pmatrix}$$

- そこから、一階の条件 $\nabla L(w, b) = \mathbf{0}$ を次の形に式変形しよう

$$\begin{pmatrix} ? \\ ? \end{pmatrix} \begin{pmatrix} w \\ b \end{pmatrix} = \begin{pmatrix} ? \end{pmatrix}$$

線型単回帰の場合 > 最小二乗法 > 最適化手法

実際に最適なパラメーターを求めてみよう

- 最小化問題「 $\text{Min}_{(w,b) \in \mathbb{R}^2} L(w,b)$ 」の一階の条件

$$\frac{\partial L(w,b)}{\partial w} = 0, \quad \frac{\partial L(w,b)}{\partial b} = 0 \Leftrightarrow \begin{cases} (1/n) \sum_i 2(wx_i + b - y_i)x_i = 0 \\ (1/n) \sum_i 2(wx_i + b - y_i) = 0 \end{cases}$$

- これを解いて、解 $(\hat{w}, \hat{b})$ は、

$$\begin{pmatrix} \hat{w} \\ \hat{b} \end{pmatrix} = \begin{pmatrix} \sum_i x_i^2 & \sum_i x_i \\ \sum_i x_i & n \end{pmatrix}^{-1} \begin{pmatrix} \sum_i x_i y_i \\ \sum_i y_i \end{pmatrix}$$

線型単回帰の場合 > 最小二乗法

ここまですを一度まとめておく

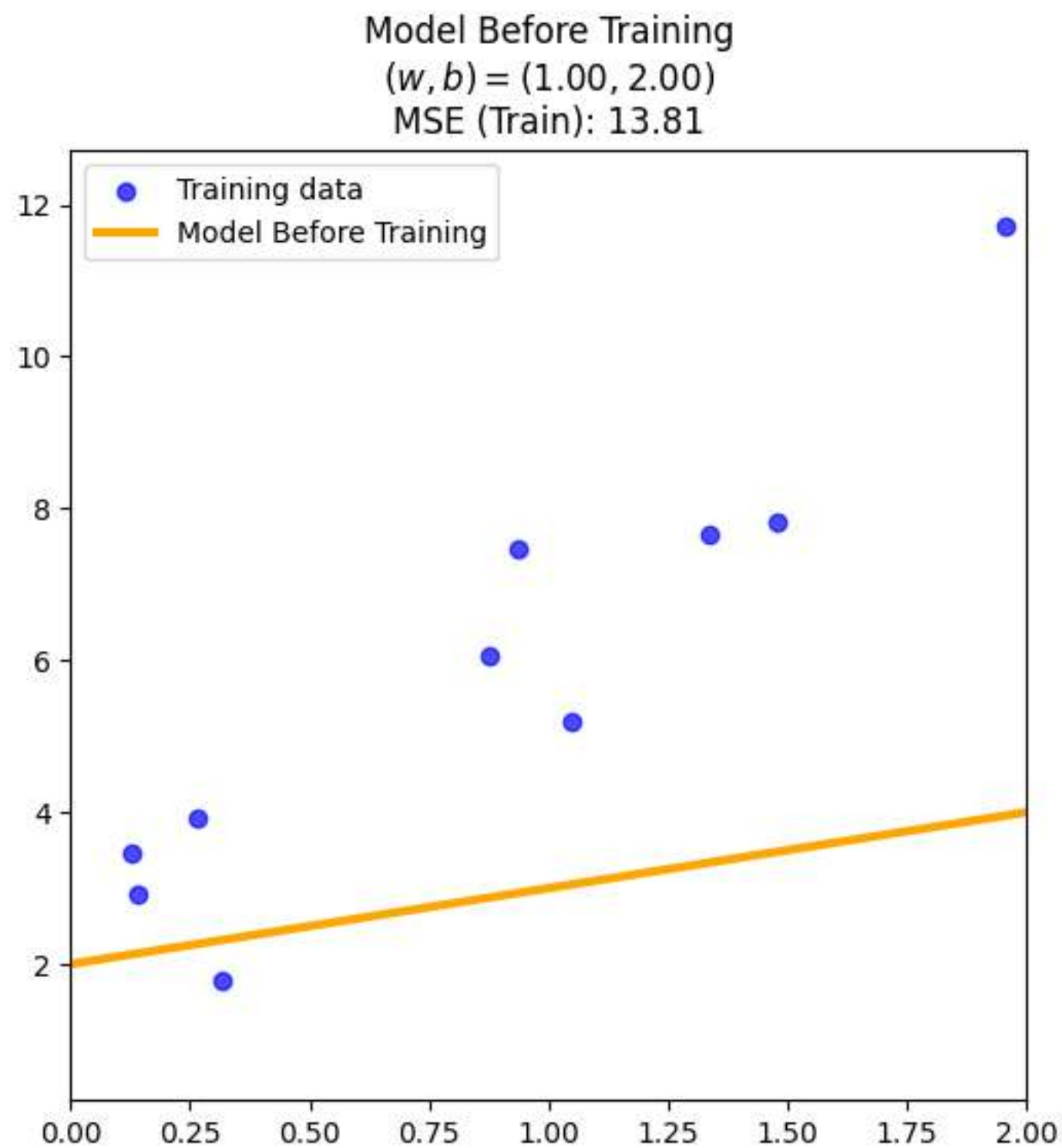
- 回帰タスクのための、最小二乗法による教師付き学習（線型単回帰の場合）
 - モデルは $f_{w,b}(x) = wx + b$ ($w, b \in \mathbb{R}$) とする.
 - パラメーターは、目的関数 $L(w, b) := \frac{1}{n} \sum_{i=1}^n (y_i - f_{w,b}(x_i))^2$ の最小化により選ぶ.
 - 今回の場合、最適解は、一階の条件により、次の式で求められる

$$\begin{pmatrix} \hat{w} \\ \hat{b} \end{pmatrix} = \begin{pmatrix} \sum_i x_i^2 & \sum_i x_i \\ \sum_i x_i & n \end{pmatrix}^{-1} \begin{pmatrix} \sum_i x_i y_i \\ \sum_i y_i \end{pmatrix}$$

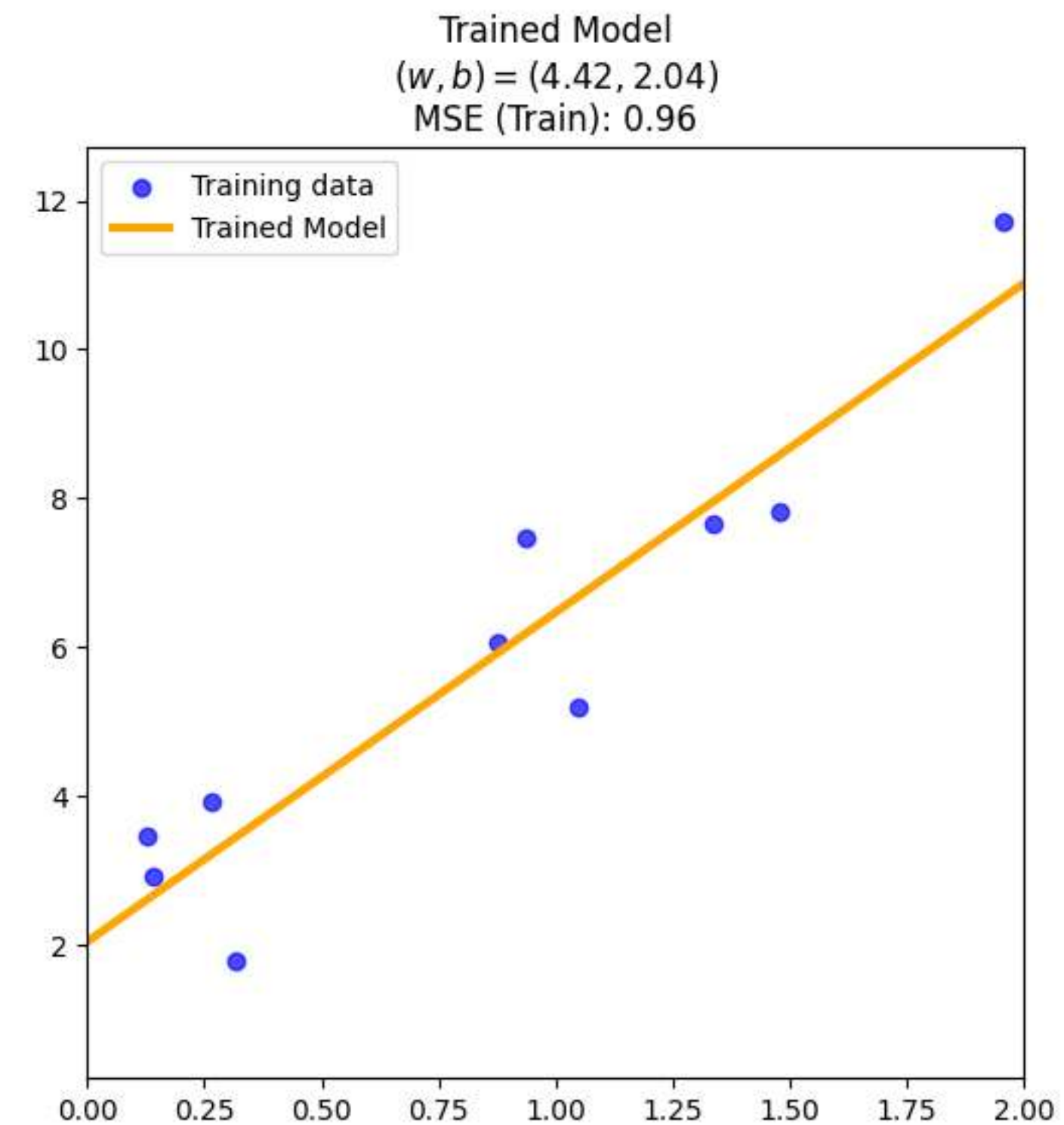
- 訓練データから $(\hat{w}, \hat{b})$ を計算し、 $f_{\hat{w}, \hat{b}}(x) = \hat{w}x + \hat{b}$ を訓練済み予測器として使用する

線型単回帰の場合 > 最小二乗法 > 訓練の実行例

左：訓練前の予測器， 右：訓練後の予測器（青点が訓練データ）



最小二乗法で
パラメーターを
調整



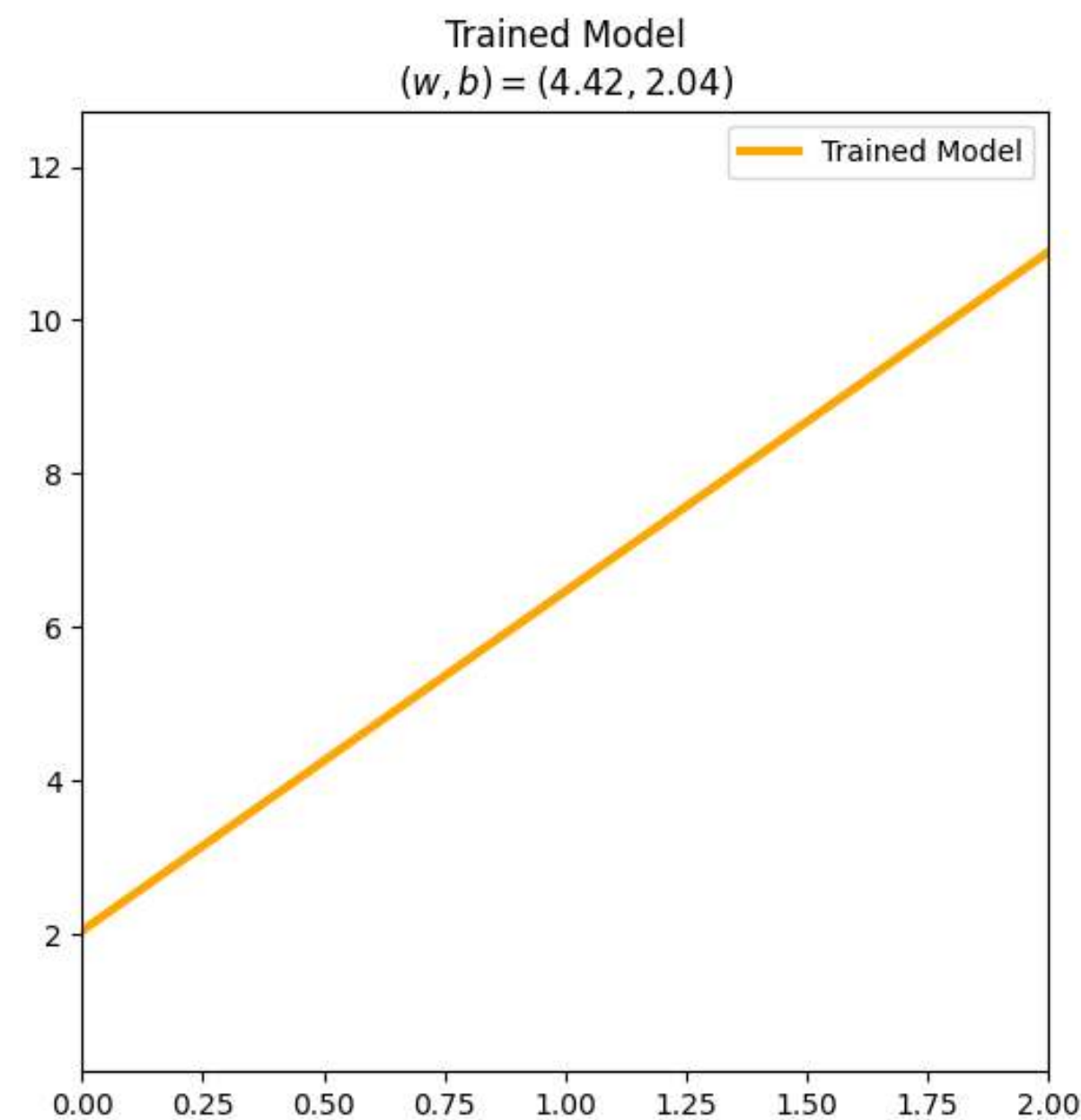
ランダムに決めたパラメーター

最小二乗法で得たパラメーター

線型単回帰の場合 > 最小二乗法 > 訓練済みの予測器

とりあえず予測器を学習することまではできた

- しかし、ゴールは、未知データに対する性能が良い予測器を手に入れること
- この訓練済みモデルで、未知データでの予測は、どれくらい当たるのか？



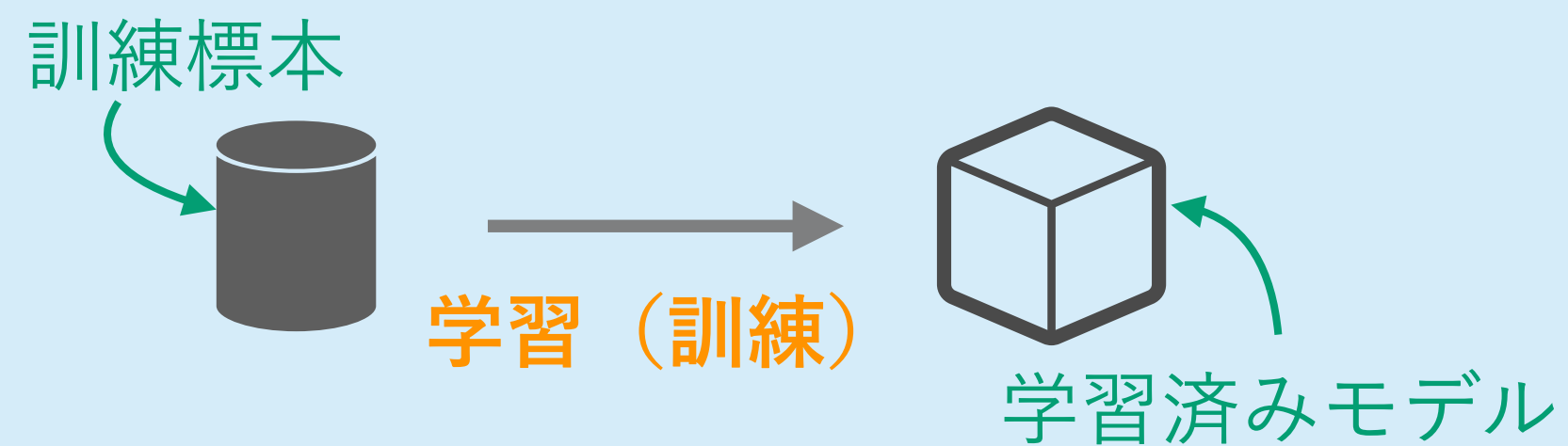
訓練済みモデル

訓練・評価・運用＞用語

典型的な機械学習手法は、2つのフェーズ（段階）からなる

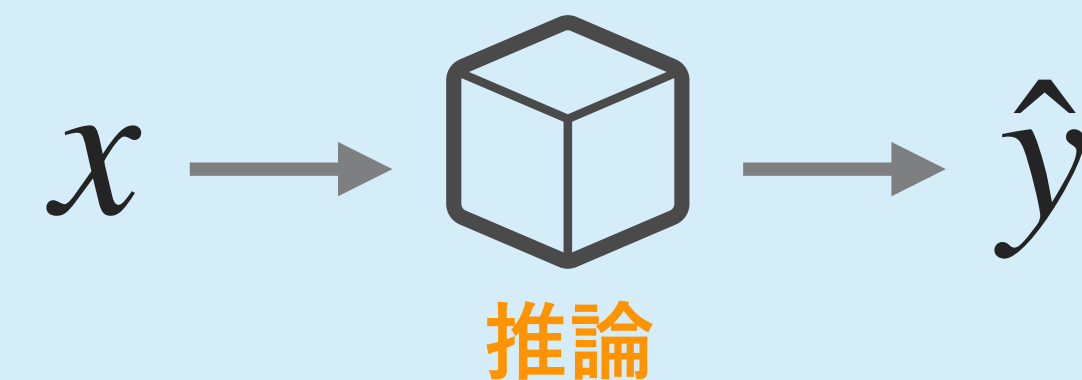
訓練フェーズ（学習フェーズ）

- データをもとにモデルを訓練
 - パラメーターを調整（学習）
 - 学習したパラメーターを記録（ファイルに保存）しておく



推論（inference）フェーズ

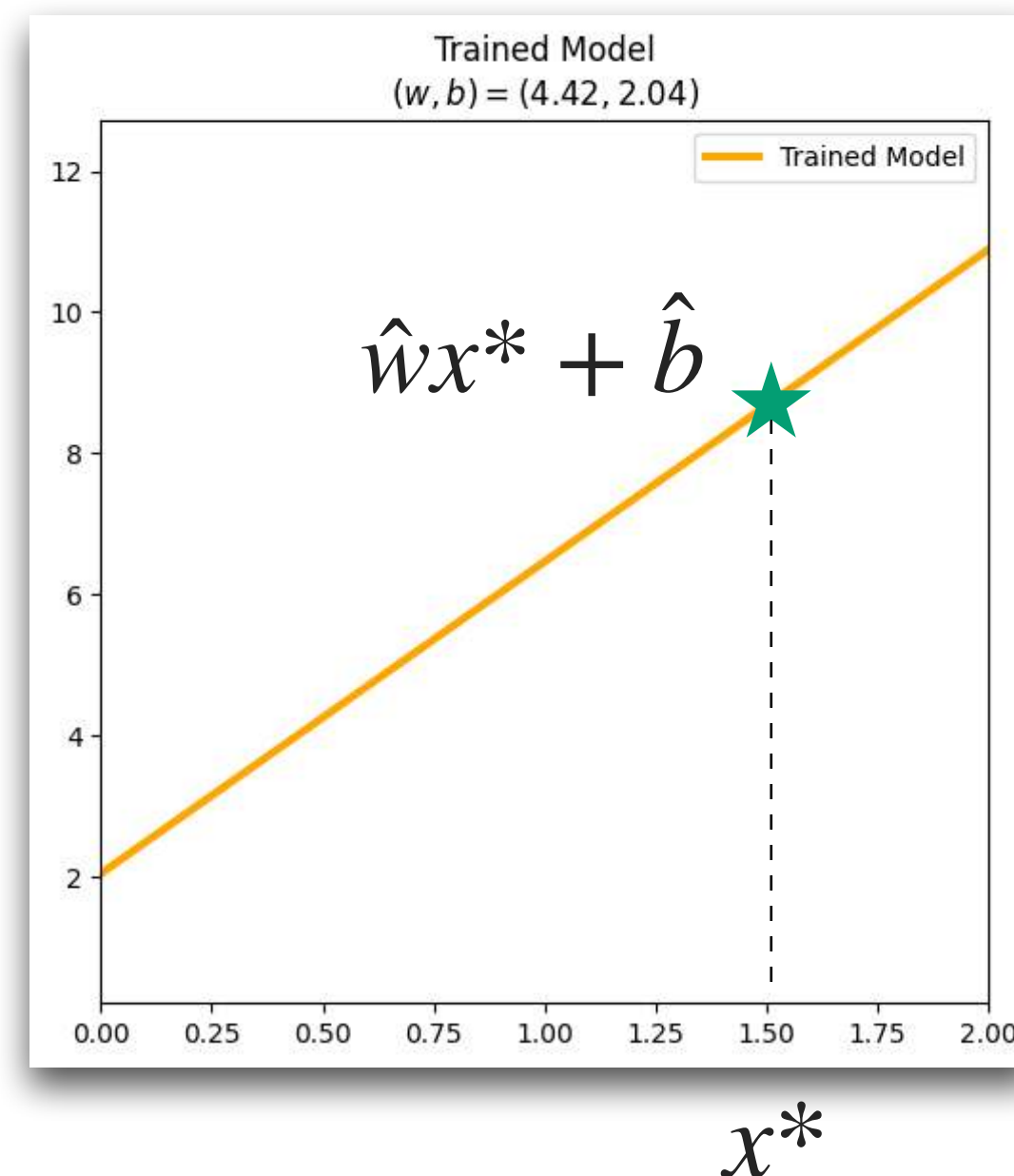
- 訓練済みモデルを実際に使用
 - 新たな入力 x が渡される
 - 学習済みモデルを用いて予測



線型単回帰の場合＞推論フェーズ

推論フェーズでは，学習済みモデルを予測器として使用

- 具体的には，学習によって得たパラメーターを使ってモデルで予測を出す
 - 新たなデータ x を渡されるたび，学習した $(\hat{w}, \hat{b})$ を使って $f_{\hat{w}, \hat{b}}(x)$ で予測

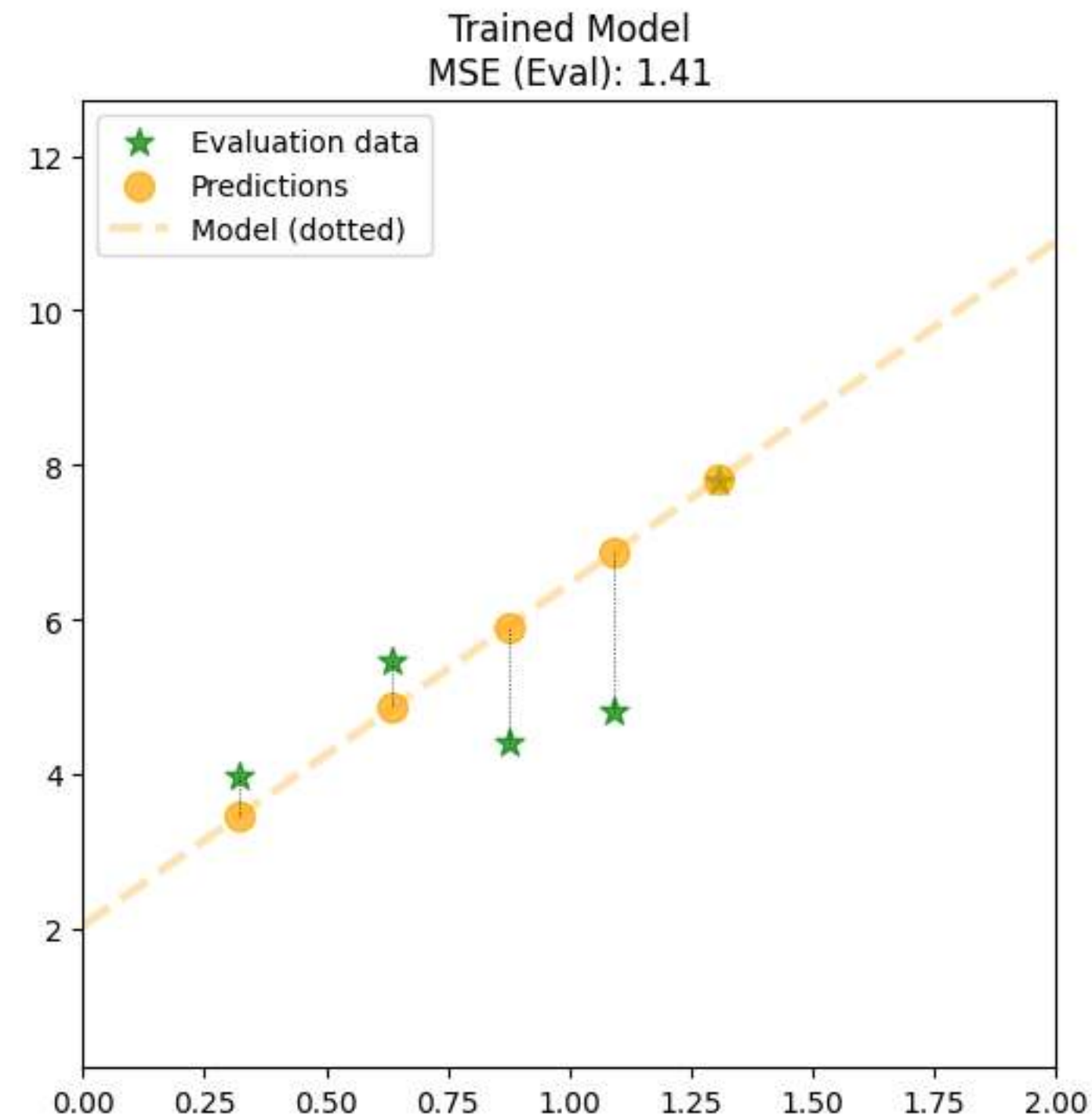


$$f_{\hat{w}, \hat{b}}(x) = \hat{w}x + \hat{b}$$

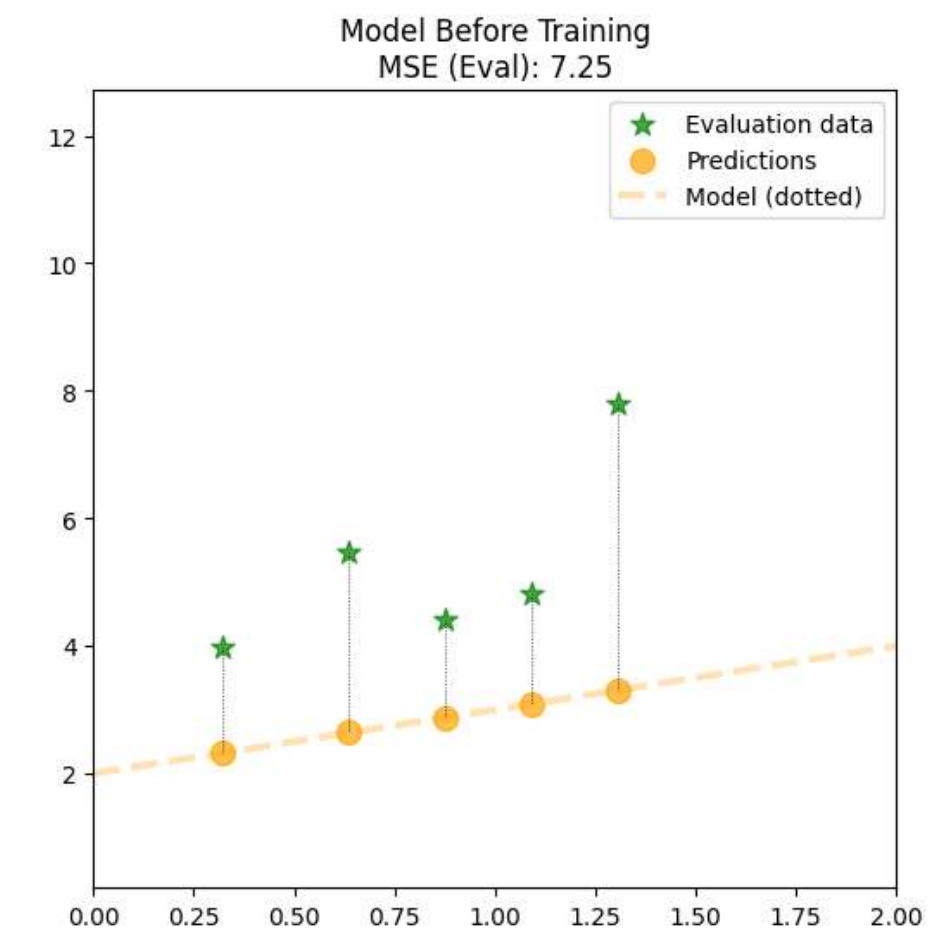
線型単回帰の場合＞推論フェーズ

今回は人工データなので新たにデータを生成して推論～評価

★：正解ラベル y
●：予測ラベル $\hat{y}$



学習後のモデルによる、
新規データにおける予測結果



(参考)

学習前のモデルによる予測結果

学習したモデルの評価・運用

訓練・評価・運用

たとえば、動画コメントのフィルタリングを考えてみよう

- あなたが運営する動画配信サイトに、不適切なコメントを自動で非表示（保留）にする機能を付け加えたいとする。



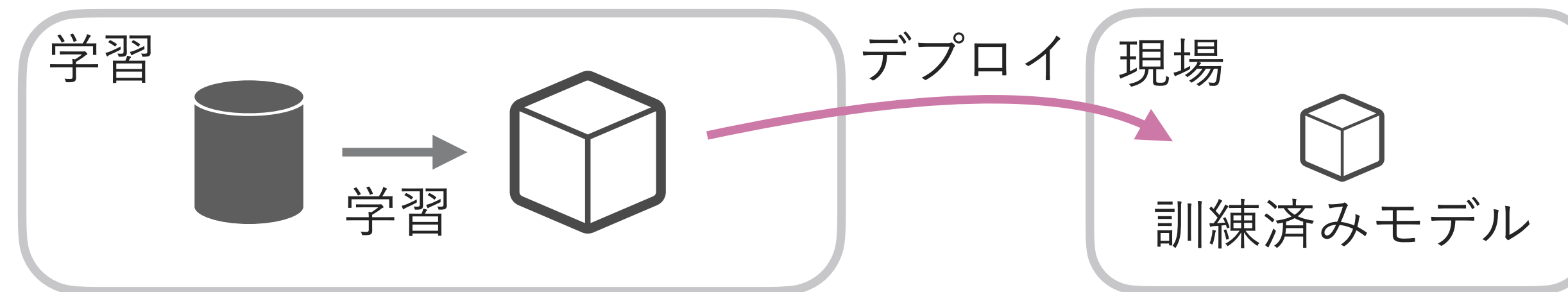
このフィルターを作りたい

- 学習時にすること：
 - 過去のコメントを集め、そこに人間が「不適切」「適切」とラベルを付けて、教師付き学習によって、良さそうな予測器を得る。

訓練・評価・運用

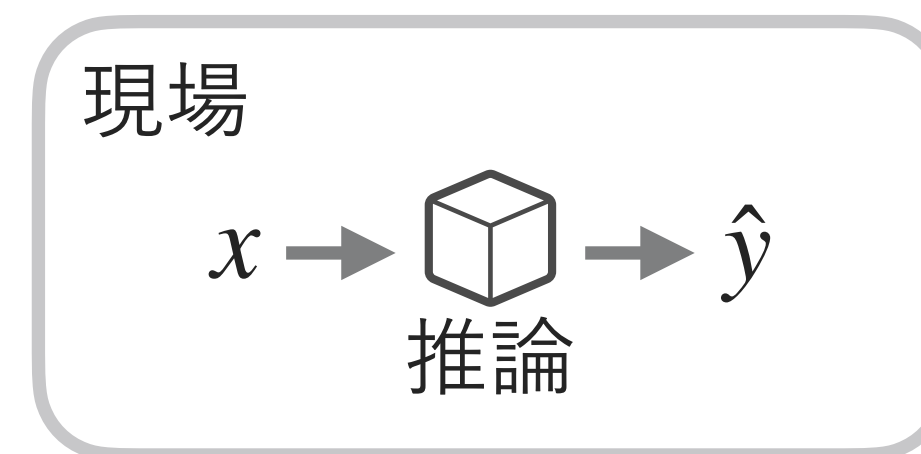
学習したモデルは、それを使う現場にデプロイ（deploy, 配備）する

- 学習後にすること：モデルを、サイトのサーバー上で使える状態にしておく



- 実際の運用：

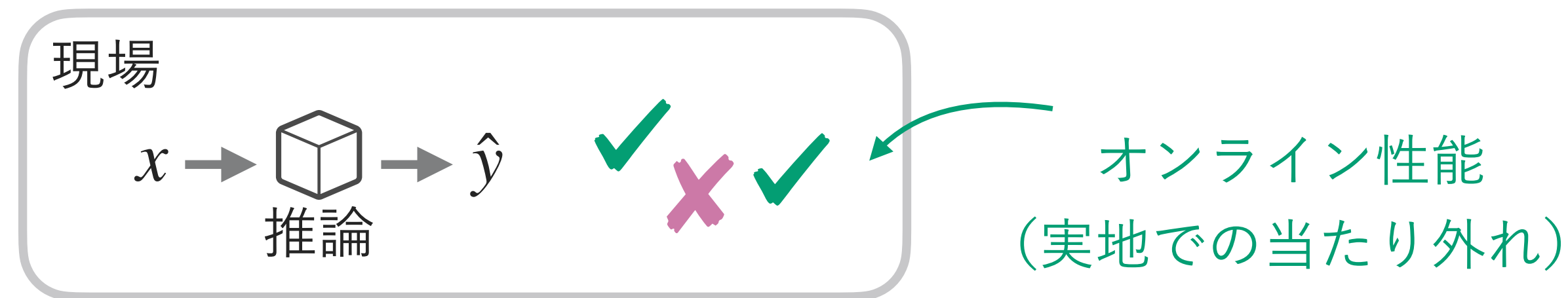
- サイト上で新たなコメントが届くたび、作った予測器にその都度予測をさせ、「不適切」という予測であれば表示せずに隠す



訓練・評価・運用

デプロイ後にも当然、予測の当たり外れが生じる

- 実運用における予測器の性能を**オンライン性能 (online performance)** という



- 運用中のモデルのオンライン性能は、頑張って計測しなければならない
 - AIによる不適切判定の答え合わせには、コメント欄の内容を事後調査して、「適切」「不適切」のラベルを付けていくしかない



費用もかかる
一時期ではなくずっと続く

- 余裕がなく、計測し続けられないケースもある（訓練済モデルを信じて使う）

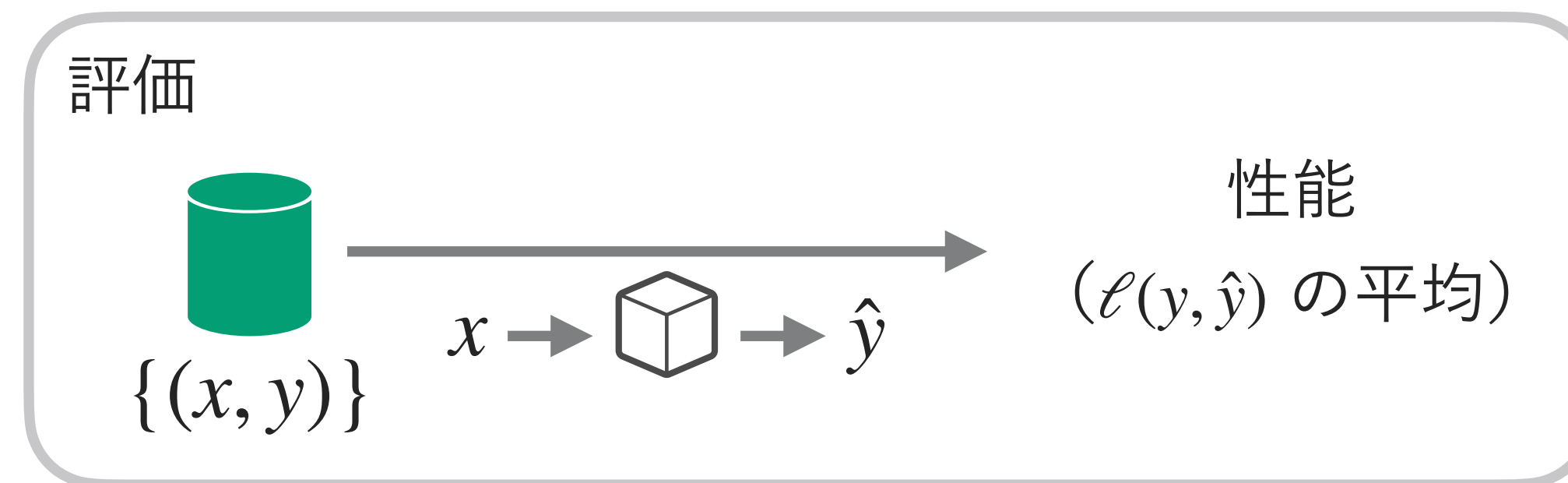
【補足】 応用によっては、オンライン性能の計測がもっと簡単な場合もある（クリック予測など）が、意図して追いかけていなければ計測できないことは同じ。

【用語】 ここでのオンラインは「インターネット上の」という意味ではない。「本番システムに配線されている」ことの比喩で「オンライン」と慣習的に呼ぶ。

訓練・評価・運用

当然，デプロイ前にある程度，性能の良し悪しの見当をつけておきたい

- オンライン性能は（定義上）実際に使ってみるまでわからないので，本当の意味でオンライン評価の代わりにまではならないが，デプロイの前にあらかじめ性能を評価しておきたい
 - ユーザーに迷惑をかけないため・オンライン評価が難しいなど
- デプロイ前に行う性能評価のことを**オフライン評価（offline evaluation）**という



【用語】 オンライン評価・オフライン評価という用語は，機械学習の教科書や論文では基本的にあまり出てこない．どちらかということとその応用分野にあたるデータサイエンスでの用語である．いずれにせよ，実運用での性能と，手持ちのデータにおけるベンチマーク的な性能を，概念として区別しておくことは有用である．

【補足】 実際のところ「オフラインで良くてもオンラインで振るわない」現象は多く見られる（原因は様々）ので，あくまでオンライン性能の計測は重要である．

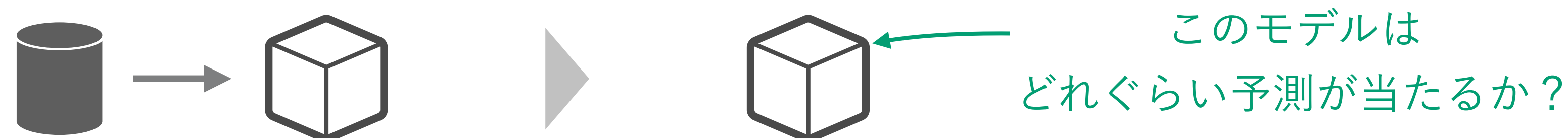
訓練・評価・運用＞オフライン評価

オフライン評価はどうか？

- 人工データでもない限り，実際に新規データを無尽蔵に生み出すことはできない



- それなので，手持ちのデータで，学習に加えて性能評価まで行う必要がある

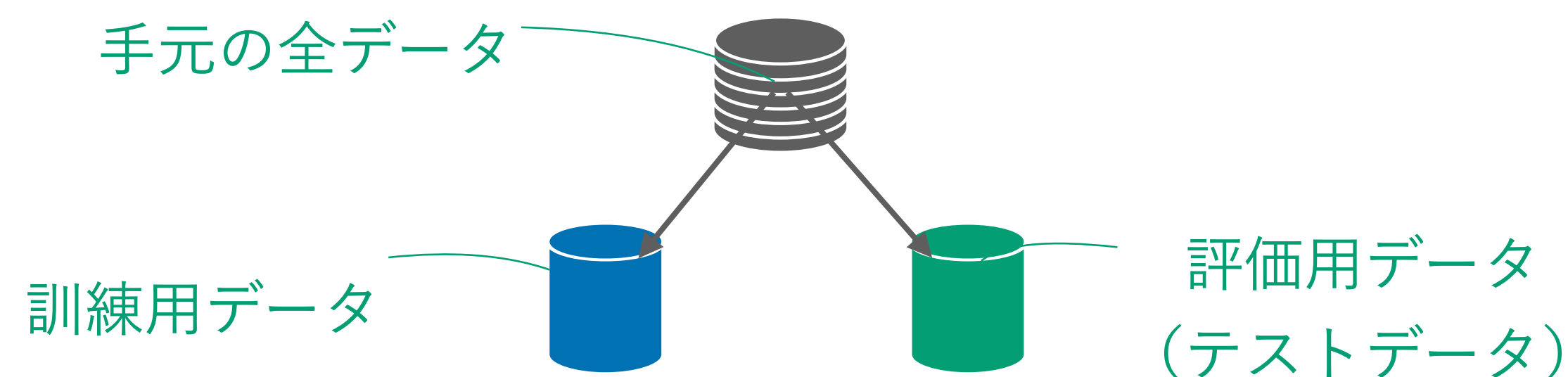


【補足】性能評価をせずに，学習の成功を信じて思い切って使ってみることもできるが，「本当にこれ使って当たるんですか？」と問われたら困ってしまう．自分が作ったモデルの予測性能を定量的に評価して知っておくことは，多くの場合においてモデル作成者の責任の一部であるし，関係者からも求められるため重要である．

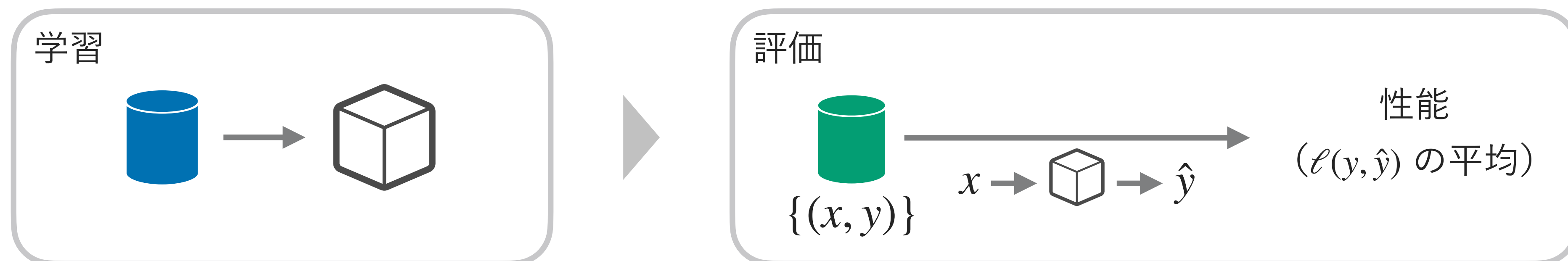
訓練・評価・運用＞オフライン評価

ここで、手持ちの全データを学習に用いず、評価用データを分けることが重要

- 手持ちのデータをあらかじめ、訓練用と評価用（テスト用）に分割



- 訓練用データだけでモデルの学習を行い、評価用データで予測性能を評価する

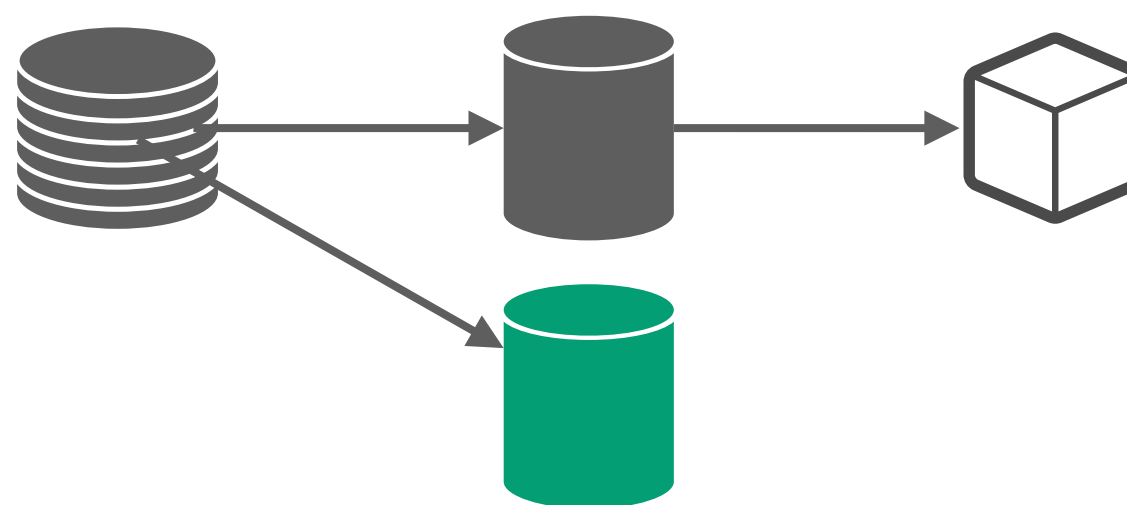


【補足】先ほどの学習済みモデルは、実は予めこのようにランダムに分割した訓練用データで学習した結果を見せていた。

訓練・評価・運用＞オフライン評価

なぜ評価用データを分けるのか？説明の言い回しはいくつかある

- 実用場面では「訓練標本と独立な，未知のデータ」が出てきて推論（予測）する
 - 実際の使用場面をなるべく模倣して，なるべく近い性能を推定したいため
 - 学習に使ったデータを評価にも使い回してしまうと，評価が甘くなってしまう
- 同じデータを使い回すと，学習時にテストをカンニングしているようなもの
 - 一般的な規則性を見出していなくてもデータを丸暗記すれば答えられてしまう
 - それができないように，テストデータをあらかじめ分ける



訓練・評価・運用＞オフライン評価

学習に使ったデータで予測の当たり具合を見ても適切な評価にはならない

- 言い換えると、学習後の訓練損失は、

$$\hat{R}(f_{\hat{\theta}}) := \frac{1}{n} \sum_{i=1}^n \ell(y_i, f_{\hat{\theta}}(x_i))$$

- リスク（期待損失）

$$R(f_{\hat{\theta}}) := \mathbb{E} [\ell(Y, f_{\hat{\theta}}(X))]$$

に対する良い推定値にならないため

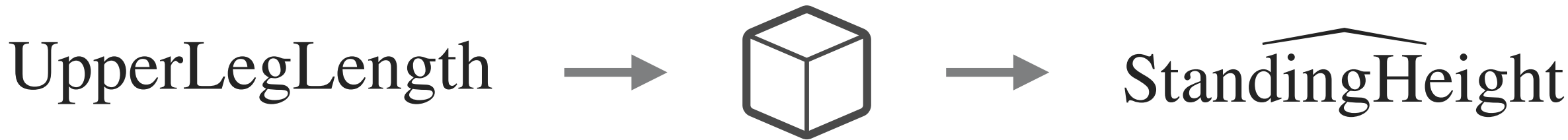
訓練・評価・運用＞実験データ

NHANES（エヌ・ヘインズ）身体測定データで具体的に見てみよう

- 2017–2018年に収集された、米国の人口における、体重や身長などの測定データ

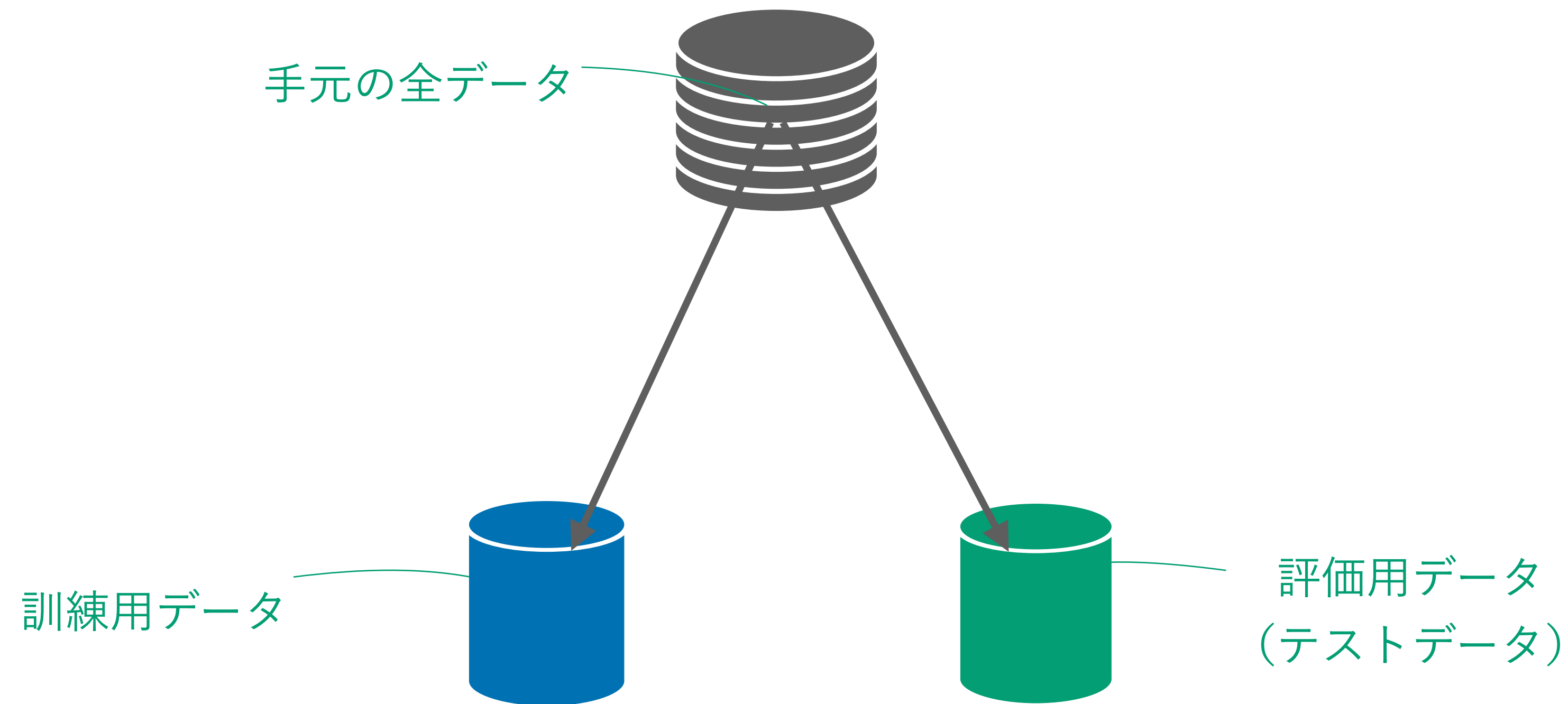
UpperLegLength	StandingHeight
37.0	158.3
46.6	175.7
38.1	158.4
31.8	150.2
31.9	151.1

- このデータで、大腿部の長さ（upper leg length, 太もも）から身長を予測する予測器を作ってみよう



訓練・評価・運用＞実験データ

まずはデータを分けておく（今回は，20%をテストデータとして無作為分割）



訓練・評価・運用＞実験データ

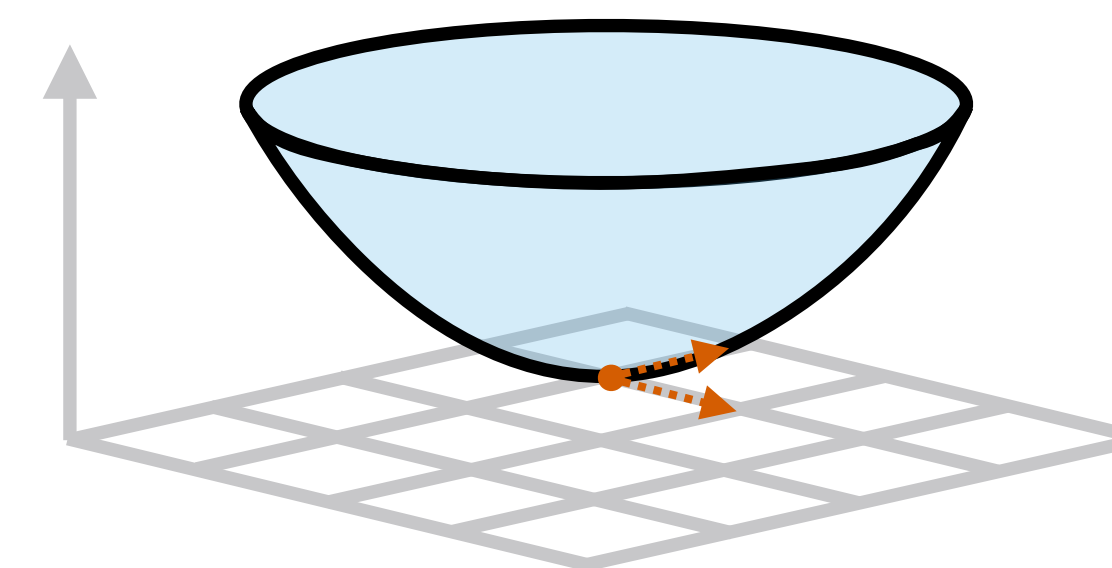
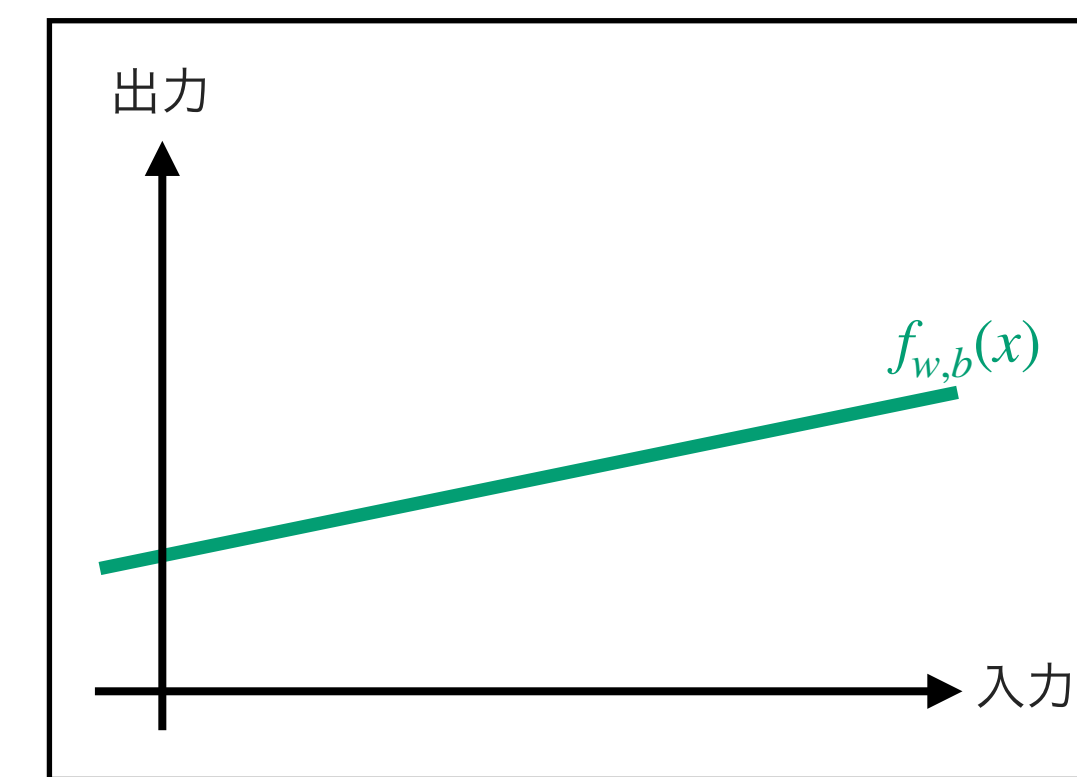
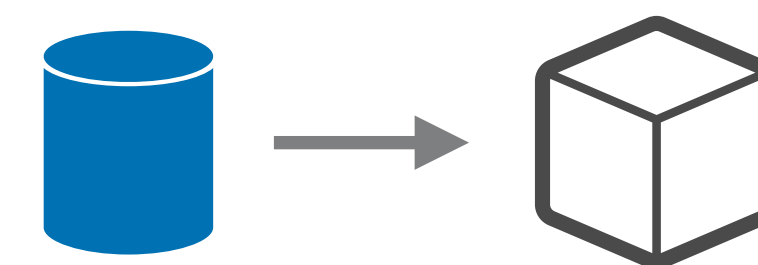
作り方（学習の方法）はここまでの話と同じ

- モデルは1次多項式 $f_{w,b}(x) = wx + b$ ($w, b \in \mathbb{R}$)
- 損失関数は二乗損失 $\ell(y, y') = (y - y')^2$
- つまり，リスクは期待二乗誤差で，

$$R(f_{w,b}) := \mathbb{E} [\ell(Y, f_{w,b}(X))]$$

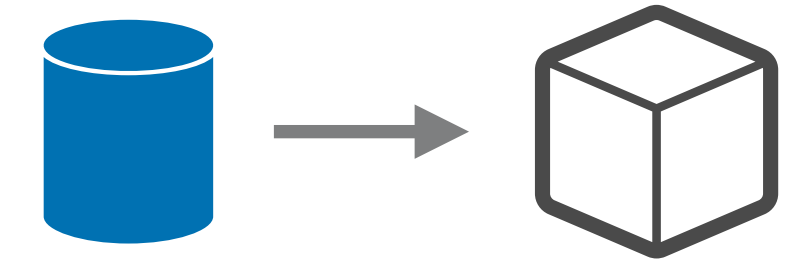
- 最小二乗法（ordinary least squares）により学習

$$\text{Min}_{(w,b) \in \mathbb{R}^2} \frac{1}{n} \sum_{i=1}^n (y_i - wx_i - b)^2$$



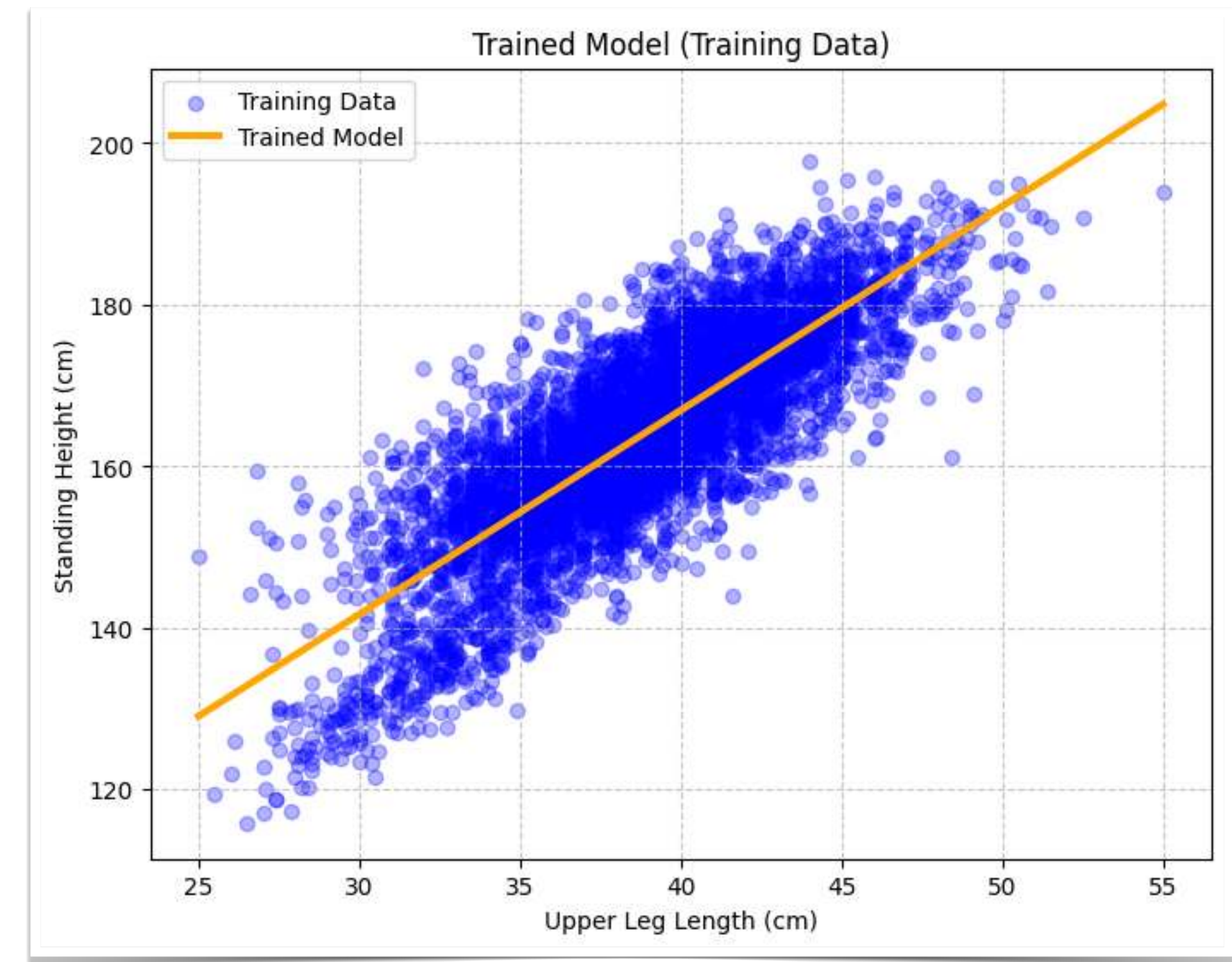
訓練・評価・運用＞訓練済み予測器

作り方（学習の方法）はここまでの話と同じ



適当に決めた予測器

$$100.00 + 2.00 \cdot \text{UpperLegLength}$$



学習した予測器

$$65.98 + 2.52 \cdot \text{UpperLegLength}$$

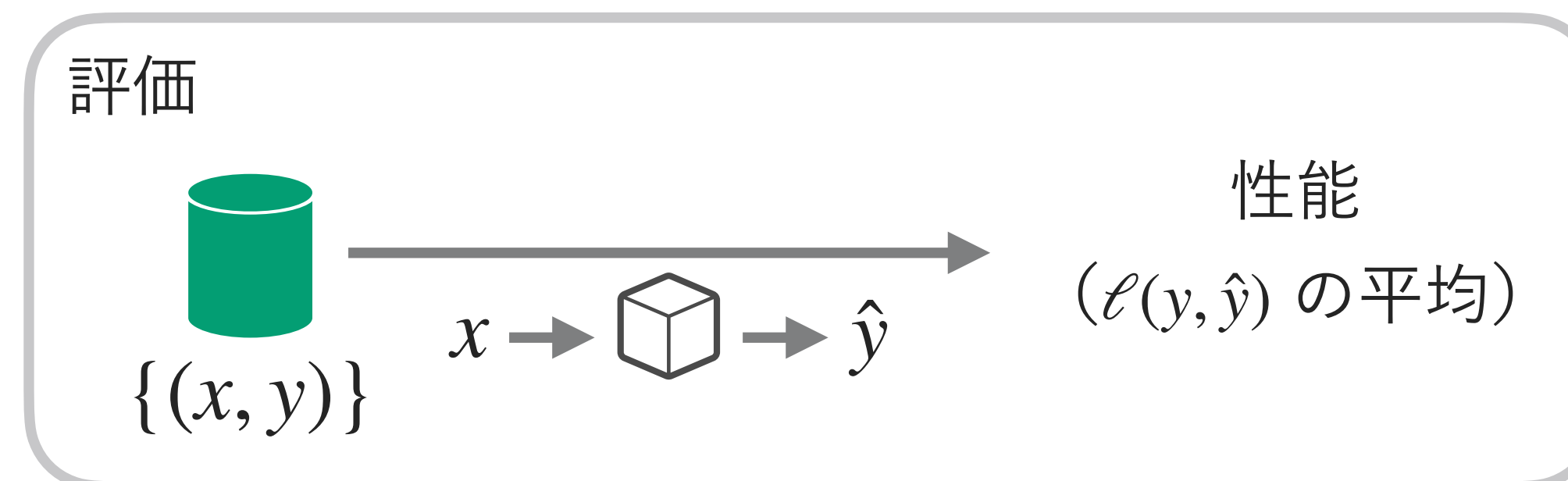
訓練・評価・運用＞訓練済み予測器

訓練済みモデルは，このようになった

- 果たしてこれは，どのくらい当たると思われるか？

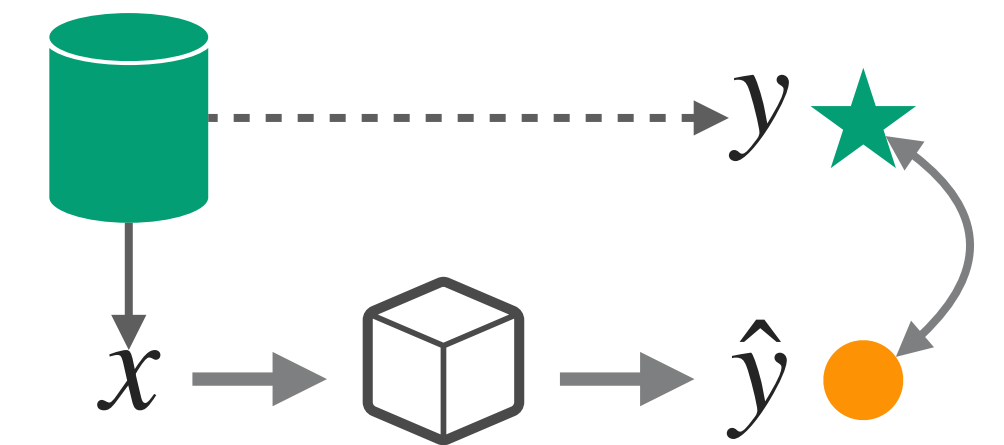
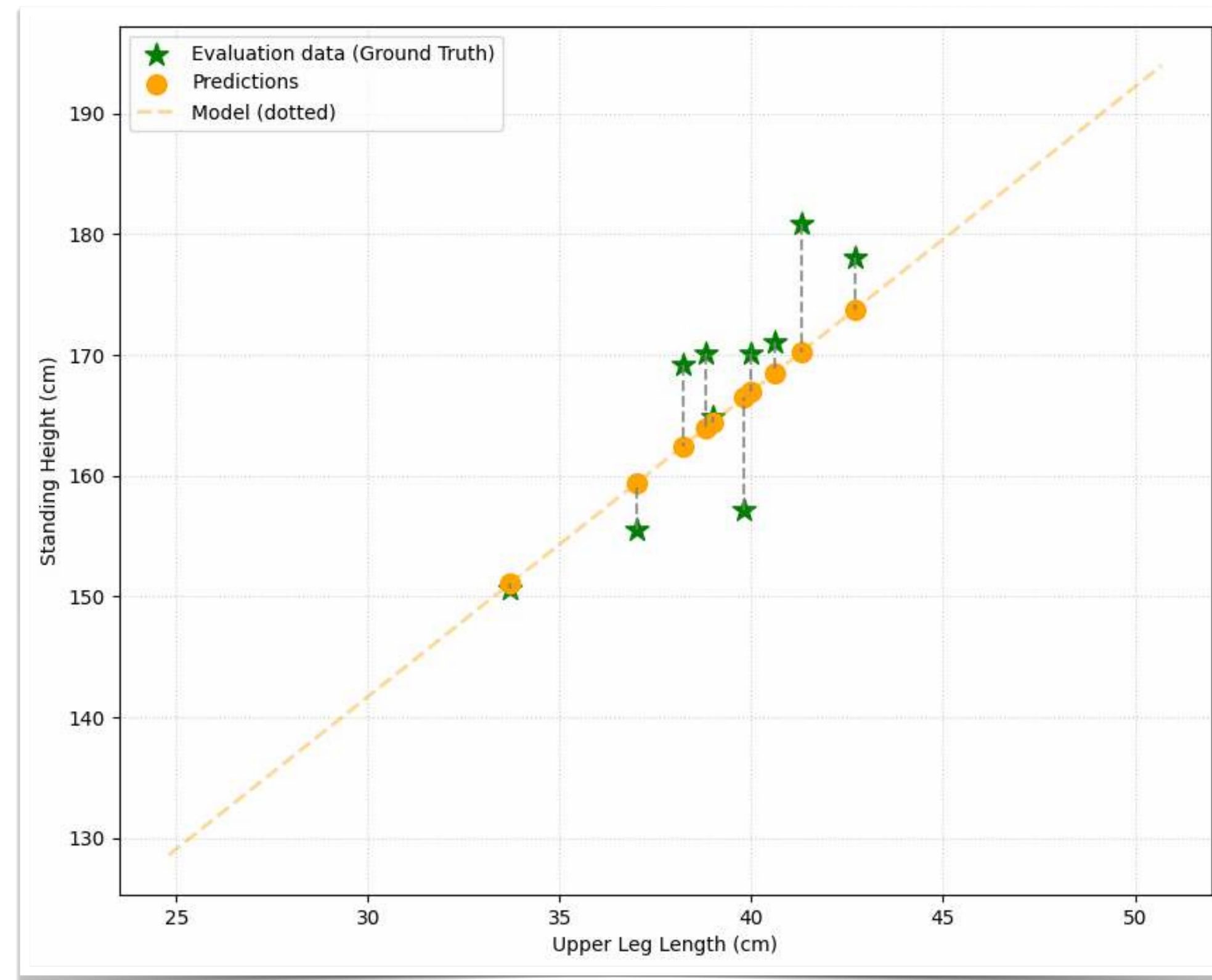
$$\widehat{\text{StandingHeight}} = 65.98 + 2.52 \cdot \text{UpperLegLength}$$

- 事前分割したテストデータで，オフライン評価を試みよう



訓練・評価・運用＞オフライン評価

事前に分割したテストデータで、オフライン評価を行った結果 $\{(x, y)\}$



■ 平均二乗誤差 (MSE) は **53.8358**, RMSE ($=\sqrt{\text{MSE}}$) でいうと: **7.3373**

【参考ソースコード】 [番号]_simple_linear_ols_regression_NHANES.ipynb 【補足】 ここで $\text{MSE} := \frac{1}{n} \sum_{i=1}^n (\text{StandingHeight}_i - \widehat{\text{StandingHeight}}_i)^2$.

訓練・評価・運用＞オンライン評価

実用性能は、皆さん自身が使ったときの性能

- 訓練済みモデルがどの程度の誤差で予測に成功するか

$$f(\text{UpperLegLength}) = 65.98 + 2.52 \cdot \text{UpperLegLength}$$

- 測り方

足の付け根の真ん中あたり、
骨盤から測る

膝のお皿の手前側までを測る



椅子などに座り、膝は90度

足の筋肉に沿わず直線的に
骨の長さを測るつもりで測る

- オフライン評価の結果からすると、平均的には 7.3cm 程度の誤差が見込まれる

【参考】 3.4.5 of National Health and Nutrition Examination Survey (NHANES). (2017, January). Anthropometry Procedures Manual. (https://wwwn.cdc.gov/nchs/data/nhanes/public/2017/manuals/2017_Anthropometry_Procedures_Manual.pdf)

訓練・評価・運用＞まとめ



（オフライン）評価データは学習データと分ける必要があるのが原則

- 実際に，訓練済みモデルのオンライン性能を評価してみよう：
 - 自宅で計測 → 学習済みモデルで予測 → 誤差を計算 → 報告
 - 課題（小テスト）のフォームから，結果を報告してください（任意）
 - ある程度の件数が集まったら集計して，オンライン性能の評価結果を何回目かの講義で報告します
- （進んだ話題）なお，評価用データの分け方は創意工夫が必要な場合がある

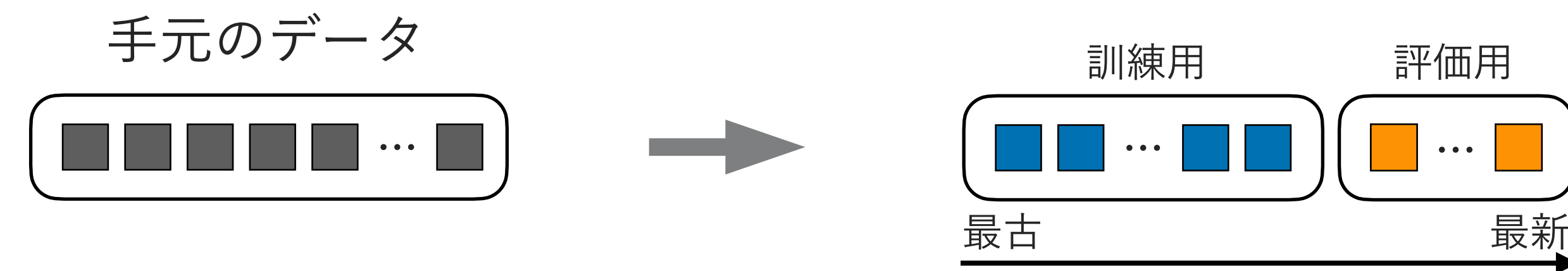
訓練・評価・運用＞補足＞置き置き法（Holdout method）

ポイントは、デプロイ後の状況をうまく模倣できる評価用データにすること

- 無作為分割（random split）（例）訓練用:評価用=7:3で置き置く



- 時系列分割（time-series split）（例）最新1ヶ月のデータを評価用に置き置く



【用語】置き置き法は、文字通り、あとで用いるデータを学習前に置き置いておく方法の総称だが、特に断りがなければ無作為分割を指すことも多い。なお、置き置き法という訳語は（優れていると思うが）浸透しているわけではない。日本語でも外来語としてホールドアウト法と呼ばれることが多い。

【補足】置き置き法以外にも、交差検証法や情報量規準など、最終的な性能を推定するための手法は数多く存在する。



線型モデル

基本知識の確認＞ベクトルの内積

同じ位置の成分同士を掛けて、足し上げた値を内積（inner product）という

■ ベクトルの内積 $\langle \mathbf{a}, \mathbf{b} \rangle$

$$\mathbf{a} = \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_d \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_d \end{bmatrix} \text{ に対して, } \langle \mathbf{a}, \mathbf{b} \rangle := a_1 b_1 + \cdots + a_d b_d$$

■ 別の表記法 $\mathbf{a}^\top \mathbf{b}$ （ $1 \times d$ 行列と $d \times 1$ 行列の積とみなして計算）

$$\mathbf{a}^\top \mathbf{b} = [a_1 \ a_2 \ \cdots \ a_d] \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_d \end{bmatrix} = a_1 b_1 + \cdots + a_d b_d$$

【補足】 $\mathbf{a}^\top \mathbf{b}$ という表記は、内積の記号 $\langle \cdot, \cdot \rangle$ を定義するまでもなく伝わる人が多いので、ある意味スマートな表記である。

基本知識の確認＞ベクトルの内積

内積の主な出番は3つ

- 重み付き和を簡潔に表す（成分ごとの重み $\mathbf{w} = (w_1, \dots, w_d)$ ）

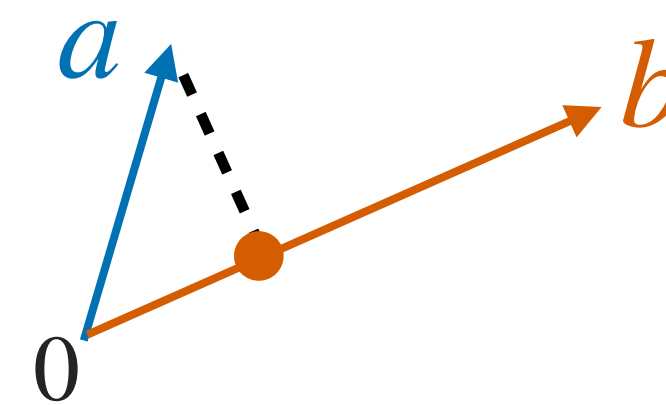
$$\mathbf{w}^T \mathbf{x} = w_1 x_1 + \dots + w_d x_d$$

- 2つのベクトルの向き of 揃い具合（同じ向きで最大，逆向きで最小）

$$\langle \nearrow, \searrow \rangle = -1, \quad \langle \nearrow, \nearrow \rangle = 0, \quad \langle \nearrow, \nwarrow \rangle = 1 \quad (\text{長さ1の場合})$$

- 特定の方向の成分の抽出（ベクトル $\mathbf{a}$ の「ベクトル $\mathbf{b}$ 方向成分」）

$$\mathbf{a} = \frac{\langle \mathbf{a}, \mathbf{b} \rangle}{\langle \mathbf{b}, \mathbf{b} \rangle} \mathbf{b} + (\mathbf{b} \text{ に直交する成分})$$



【補足】 同じ向きを向いている場合に内積が最も大きくなるという主張はコーシー・シュワルツの不等式から従う。ここでベクトルの長さは $\|\cdot\| = \sqrt{\langle \cdot, \cdot \rangle}$ で測る。

最後の $\mathbf{a} = (\langle \mathbf{a}, \mathbf{b} \rangle / \langle \mathbf{b}, \mathbf{b} \rangle) \mathbf{b} + (\mathbf{b} \text{ に直交する成分})$ という分解は、 $\langle \mathbf{a} - (\langle \mathbf{a}, \mathbf{b} \rangle / \langle \mathbf{b}, \mathbf{b} \rangle) \mathbf{b}, \mathbf{b} \rangle = \langle \mathbf{a}, \mathbf{b} \rangle - (\langle \mathbf{a}, \mathbf{b} \rangle / \langle \mathbf{b}, \mathbf{b} \rangle) \cdot \langle \mathbf{b}, \mathbf{b} \rangle = 0$ により確かめられる。

基本知識の確認＞ベクトルの内積の基本性質

内積は線型性と対称性をもつ

■ 線型性 (linearity) : $\langle \mathbf{x}, \alpha \mathbf{y} + \beta \mathbf{z} \rangle = \alpha \langle \mathbf{x}, \mathbf{y} \rangle + \beta \langle \mathbf{x}, \mathbf{z} \rangle$

- 一般に, 左側についても同時に線型 (双線型)

$$\langle a\mathbf{x} + b\mathbf{y}, c\mathbf{z} + d\mathbf{w} \rangle = ac\langle \mathbf{x}, \mathbf{z} \rangle + ad\langle \mathbf{x}, \mathbf{w} \rangle + bc\langle \mathbf{y}, \mathbf{z} \rangle + bd\langle \mathbf{y}, \mathbf{w} \rangle$$

- これは括弧の分配法則の一般化 $(a + b)(c + d) = ac + bd + bc + bd$
- 特に今回は「重みに関する線型性」という形で関係 : $(\mathbf{v} + \mathbf{w})^\top \mathbf{x} = \mathbf{v}^\top \mathbf{x} + \mathbf{w}^\top \mathbf{x}$

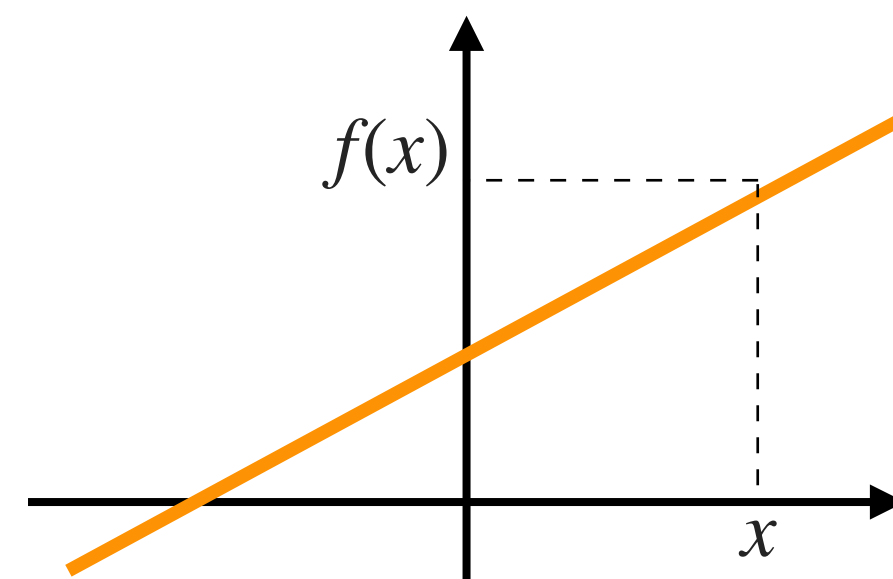
■ 対称性 (symmetry) : $\langle \mathbf{x}, \mathbf{y} \rangle = \langle \mathbf{y}, \mathbf{x} \rangle$

- 転置で表記すると少しだけ意外に見える : $\mathbf{w}^\top \mathbf{x} = \mathbf{x}^\top \mathbf{w}$

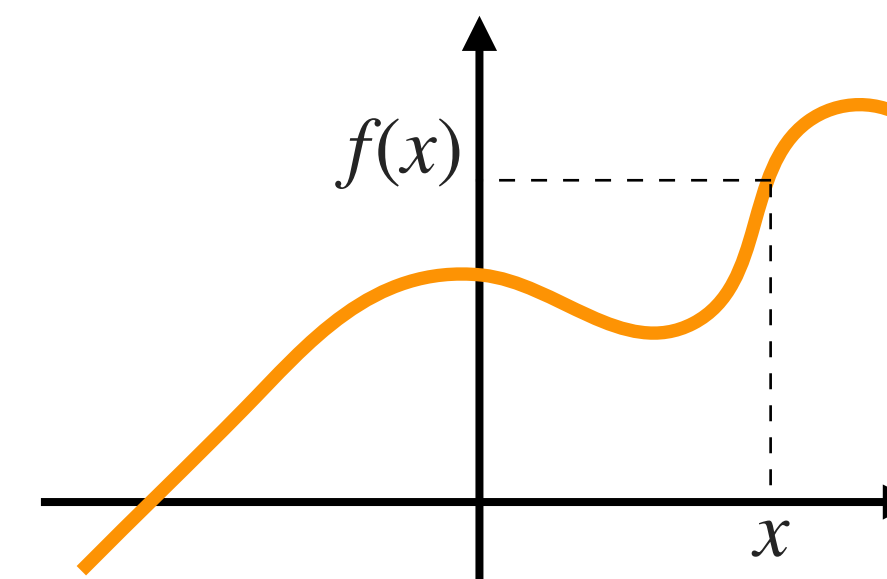
もっと複雑なモデル

もっと表現能力が高い複雑なモデルを使いたい

- 入力に対して非線型なモデルも使いたい（例：多項式回帰）



線型単回帰



非線型単回帰

- 画像／テキストなどを入力にする回帰モデルも作りたい



f_{θ}



7

キュウリの品質

“こんな映画には
滅多に出会えない”



f_{θ}



4.9

レーティング予測

線型モデル

特徴量 $\phi(x)$ とパラメーター θ の内積を使って表されるモデル

■ パラメーター線型モデル (linear-in-parameter model)

$$f_{\theta}(x) := \theta^{\top} \phi(x) \quad \left(= \sum_{b=0}^B \theta_b \phi_b(x) \right)$$

$$x \in \mathcal{X}, \phi_b(x) : \mathcal{X} \rightarrow \mathbb{R}$$

$$\phi(x) = \begin{pmatrix} \phi_0(x) \\ \vdots \\ \phi_B(x) \end{pmatrix} : \text{変換された特徴量}$$

- ここで, $\phi = (\phi_b)_{b=0}^B$ は任意の関数であり, 特徴写像 (feature map) と呼ぶ.
- $\theta \in \mathbb{R}^{B+1}$ は係数 (coefficient) や重み (weight) と呼ぶことがある.

【補足】ただし, $(\phi_b)_{b=0}^B$ は線型独立であることを仮定する. 仮に線型独立でなければ, 最初から従属なものを除外しておいても表現力は変わらない. 任意の関数と書いたが理論解析の際には多少の正則条件は課される. 【用語】 $(\phi_b)_{b=0}^B$ は基底関数 (basis functions) とも呼ばれるが, これは数学の「基底」とは違う意味なので注意.

線型モデル > 注意

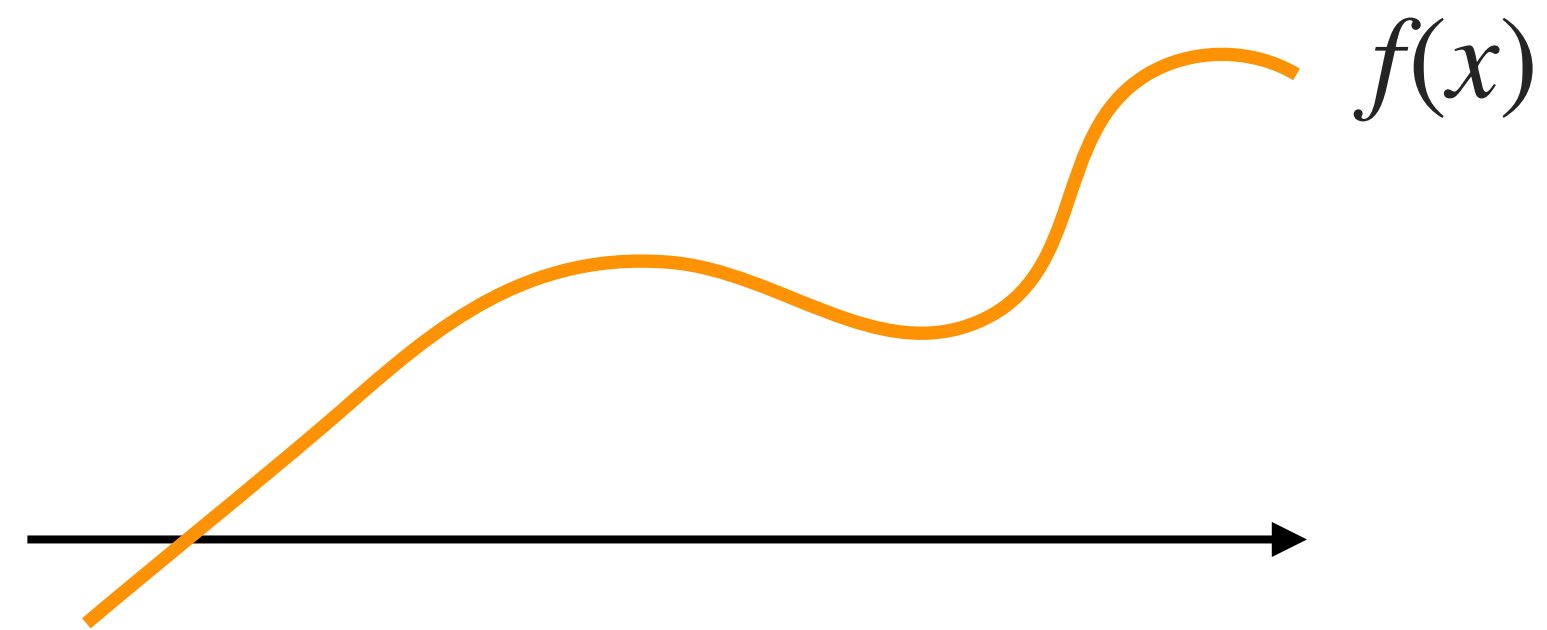
線型といってもパラメーター θ に関して線型であればよい

$$f_{\theta}(x) = \theta^{\top} \phi(x)$$

- 特徴量変換 ϕ は非線型でもよい，つまり一般に入力 x に関しては非線型.
 - 特に， x に対しても線型なモデル $f_{\theta}(x) = \theta^{\top} x$ は，データ線型モデル（linear-in-data model）と呼んで区別する
- この講義で単に線型モデルといったときは，パラメーター線型モデルを指す.

線型モデル

線型モデルの例としては、多項式モデルなどがある



- こんなに曲がっているのに線型とはどういうこと？

線型モデルの例＞多項式モデル

入力に対しては非線型でも、パラメーターに対しては線型

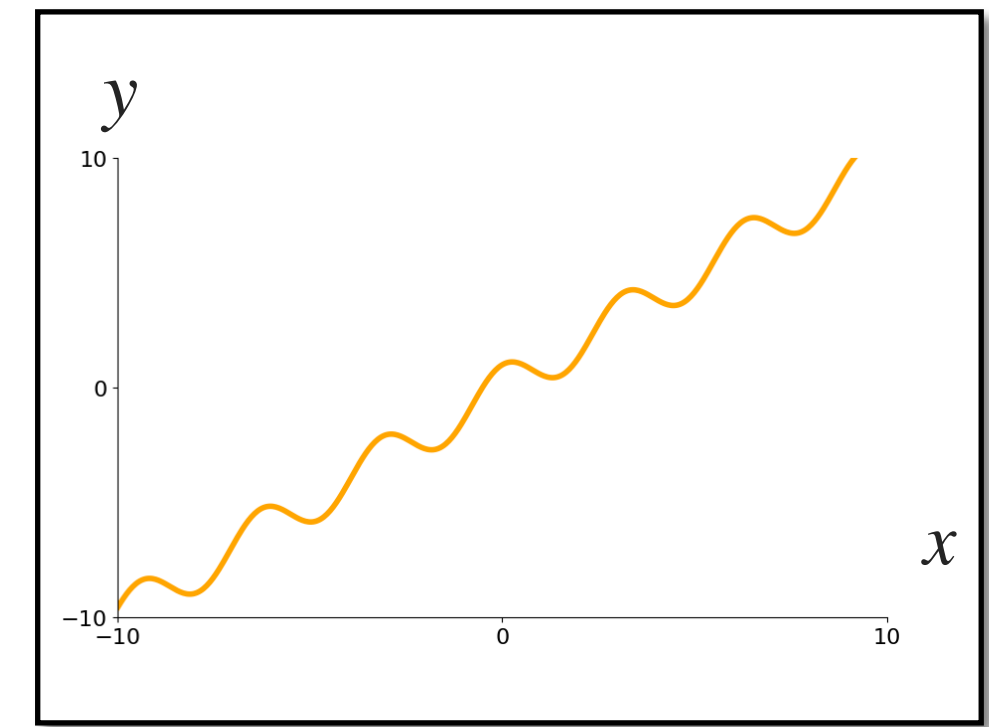
- r 次多項式モデル (r -th order polynomial model)

$$f_{\theta}(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \cdots + \theta_r x^r$$

- 多項式の次数 r を上げれば複雑なモデルを表現できる

- これは、特徴写像を次のように定義すれば、 $f_{\theta}(x) = \boldsymbol{\theta}^{\top} \boldsymbol{\phi}(x)$ と表せる

$$\boldsymbol{\phi}(x) = \begin{bmatrix} 1 \\ x \\ \vdots \\ x^r \end{bmatrix}$$



【補足】ストーン・ワイエルシュトラスの定理 (Stone-Weierstrass theorem) により、多項式モデルで任意の連続関数を近似することができる (任意の有界区間で任意の精度まで、一様ノルムでの近似が、十分に大きい k を用いれば成立)

線型モデルの例＞三角多項式モデル

色々な特徴写像がある

- 三角多項式 (trigonometric polynomial) モデル

$$\phi(x) = \begin{bmatrix} 1 \\ \sin(x) \\ \cos(x) \\ \vdots \\ \sin(kx) \\ \cos(kx) \end{bmatrix}$$

- 一定間隔で特徴量が同じ値になるので、周期性をもったモデルになる

$$f_{\theta}(x) = \theta_0 + \theta_{1s} \sin(x) + \theta_{1c} \cos(x) + \cdots + \theta_{ks} \sin(kx) + \theta_{kc} \cos(kx)$$

【補足】 ストーン・ワイエルシュトラスの定理 (Stone-Weierstrass theorem) により、三角多項式モデルで任意の連続関数を近似することができる (任意の有界区間で任意の精度まで、一様ノルムでの近似が、十分に大きい k を用いれば成立) (Powell, 1981, Theorem 13.1).

線型モデルの例＞多変数多項式モデル

色々な特徴写像がある

■ 多変数の多項式

$$\phi(x, y) := (1, \underbrace{x, y}_{\text{degree 1}}, \underbrace{x^2, xy, y^2}_{\text{degree 2}}, \dots, \underbrace{x^r, x^{r-1}y, \dots, xy^{r-1}, y^r}_{\text{degree } r})^\top$$

- あらかじめ交差項をこのように入れたおけば，多変数の多項式でも表せる

$$f_{\theta}(x, y) = \theta_0 + \theta_{1,0}x + \theta_{1,1}y + \theta_{2,0}x^2 + \theta_{2,1}xy + \theta_{2,2}y^2 + \dots + \theta_{r,0}x^r + \dots + \theta_{r,r}y^r$$

線型モデルの例＞Bag-of-words特徴量＋線型モデル

色々な特徴画像がある

- まず、語彙一覧 (vocabulary) を用意しておく

演奏, 演技, 演歌, 漢方, 漫画, ……

- 続いて、語彙に含まれる単語全ての出現回数をカウントして、特徴量として使う

入力

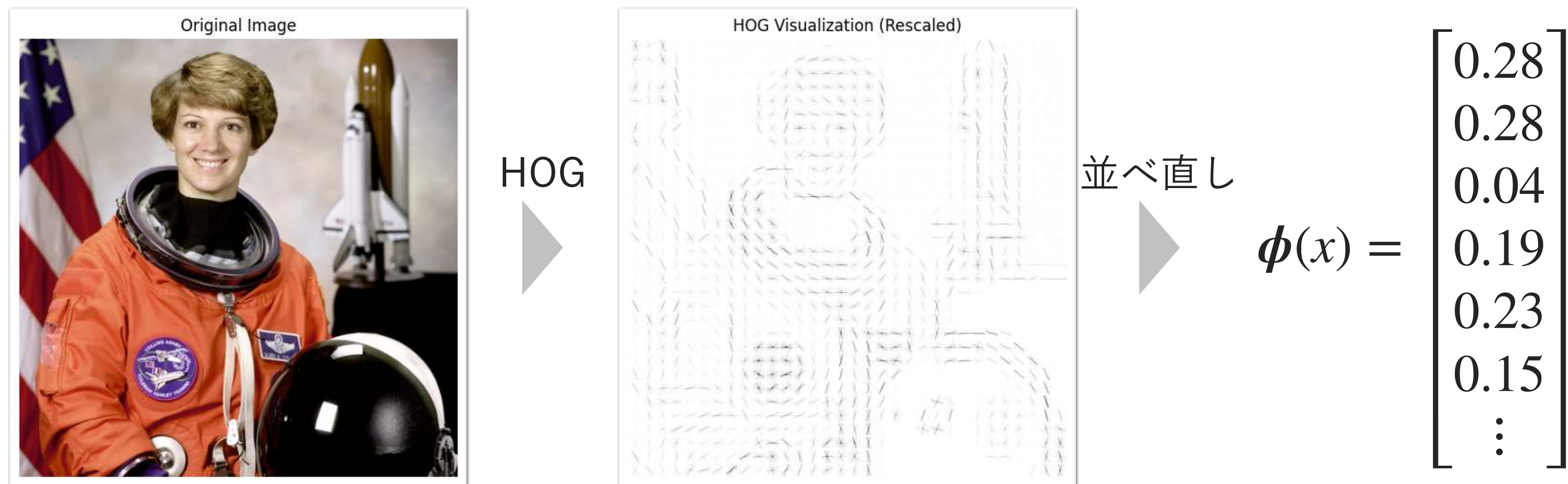
本日、老舗の演歌歌手が伝統と革新を融合させた舞台上、情感あふれる演技とともに演歌の名曲を披露し、観客から熱い拍手を浴びました。

→ {演奏: 0, 演技: 1, 演歌: 2, 漢方: 0, 漫画: 0, ……} なので, $\phi(x) = \begin{bmatrix} 0 \\ 1 \\ 2 \\ 0 \\ 0 \\ \vdots \end{bmatrix}$

線型モデルの例＞HOG特徴量＋線型モデル

色々な特徴画像がある

- HOG (Histogram of Oriented Gradients) 記述子
 - 画像の輝度勾配ヒストグラムを元にした特徴量（仕組みの説明は省略）



- 写真に人が写っているか否かの判定など

【実験コード】 [番号]_feature_map_HOG.ipynb 【画像出典】 <https://www.flickr.com/photos/nasacommons/16504233985/>

【参考】 Dalal, N., & Triggs, B. (2005). Histograms of oriented gradients for human detection. 2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05), 1, 886–893 vol. 1. <https://doi.org/10.1109/CVPR.2005.177>

特徴量エンジニアリング



特徴量変換器を実際の応用に合わせて作ることは非常に重要

- 特徴量変換の作り込みを特徴量エンジニアリング (feature engineering) という
 - 応用ごとに作り込みを行うもの。性能にも寄与することがよくある。
 - 職人芸な面がある
- 一方、自前の特徴量変換器を作らなくてもよい場合が増えている
 - 事前学習済みの特徴量抽出器をダウンロードして使えば、基本的なタスクは解けることがある

予測タスクにおける線型モデルの恩恵



パラメーター線型モデルは扱いやすい道具として今でも使われることがある

■ 線型だと何が良いか？

- 理論的な取り扱いが簡単
- 最適化が簡単にできる場合も多い
- ϕ が（今のタスクを解くために）優れていれば、これだけで十分な表現力あり

■ 弱点

- モデルの性能限界がすべて ϕ に懸かっているので、 ϕ によっては高性能が出ない

線型モデルの最小二乗法

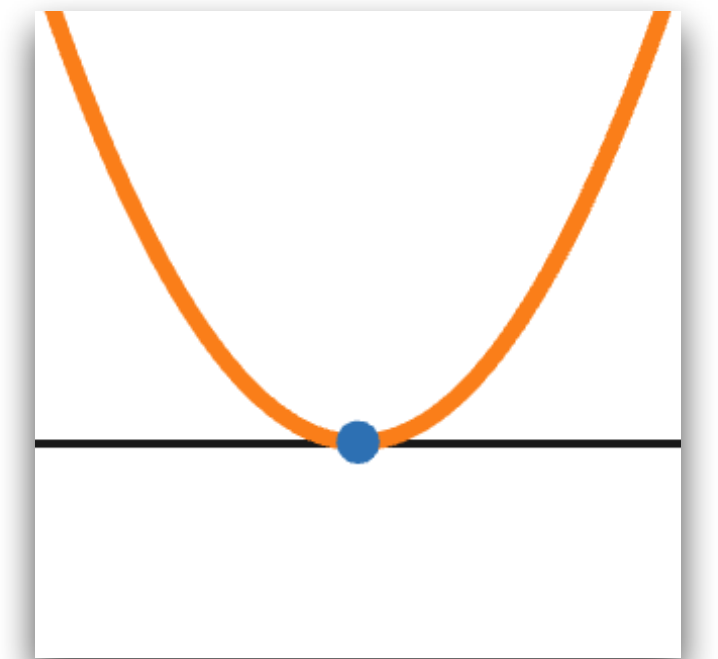
線型モデルでの最小二乗法の解

一般の線型モデルでの最小二乗法の解を求めてみよう

- 線型モデル $f_{\theta}(x) = \theta^{\top} \phi(x)$ の場合に、最適化問題

$$\text{Min}_{\theta \in \mathbb{R}^{B+1}} \frac{1}{2} \sum_{i=1}^n (y_i - f_{\theta}(x_i))^2$$

を解き、解となるパラメーター $\hat{\theta}$ を求めよ.



線型モデルでの最小二乗法の解

解の求め方の流れ

1) 目的関数は $L(\boldsymbol{\theta}) = \frac{1}{2} (\boldsymbol{\Phi}\boldsymbol{\theta} - \mathbf{y})^\top (\boldsymbol{\Phi}\boldsymbol{\theta} - \mathbf{y})$ と書き直せることを確認せよ。ここで,

$$\boldsymbol{\Phi} := \begin{pmatrix} \boldsymbol{\phi}(\mathbf{x}_1)^\top \\ \vdots \\ \boldsymbol{\phi}(\mathbf{x}_n)^\top \end{pmatrix} = \begin{pmatrix} \phi_0(\mathbf{x}_1) & \cdots & \phi_B(\mathbf{x}_1) \\ \vdots & \ddots & \vdots \\ \phi_0(\mathbf{x}_n) & \cdots & \phi_B(\mathbf{x}_n) \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix}$$

2) $\text{Min}_{\boldsymbol{\theta} \in \mathbb{R}^{B+1}} L(\boldsymbol{\theta})$ を解け。ただし, $(\boldsymbol{\Phi}^\top \boldsymbol{\Phi})^{-1}$ が存在することを仮定する。

【用語】 この $\boldsymbol{\Phi}$ を計画行列 (design matrix) とよぶ。 ヒント: $\nabla_{\boldsymbol{\theta}}(\boldsymbol{\theta}^\top \mathbf{A} \boldsymbol{\theta}) = 2\mathbf{A}\boldsymbol{\theta}$, $\nabla_{\boldsymbol{\theta}}(\mathbf{b}^\top \boldsymbol{\theta}) = \mathbf{b}$ を使う。

【参考】 自力で解いてみる場合, Petersen, K. B., & Pedersen, M. S. (2012, November). The Matrix Cookbook. Technical University of Denmark. <http://www2.compute.dtu.dk/pubdb/pubs/3274-full.html> を片手に取り組むとよい。

線型モデルでの最小二乗法の解

必要条件である一階の条件を適用する

- **勾配 (gradient)** : 偏微分を並べた縦ベクトル (パラメーター数と同じ次元)

$$\nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}) = \begin{pmatrix} \frac{\partial L(\boldsymbol{\theta})}{\partial \theta_0} \\ \vdots \\ \frac{\partial L(\boldsymbol{\theta})}{\partial \theta_B} \end{pmatrix}$$

- 最適解の必要条件として、一階の条件がある

- 制約なし最適化の場合 : $\nabla L(\boldsymbol{\theta}) = \mathbf{0}$

今回はこちら

- 制約付き最適化の場合 : KKT条件 (Karush–Kuhn–Tucker conditions)

線型モデルでの最小二乗法の解

最適解が満たす方程式（一階の条件）：勾配が0となる点を探す

$$\nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}) = \begin{pmatrix} \frac{\partial L}{\partial \theta_0} \\ \vdots \\ \frac{\partial L}{\partial \theta_B} \end{pmatrix} = \boldsymbol{\Phi}^\top \boldsymbol{\Phi} \boldsymbol{\theta} - \boldsymbol{\Phi}^\top \mathbf{y} = \mathbf{0}$$

■ これを解いて

$$\hat{\boldsymbol{\theta}} = (\boldsymbol{\Phi}^\top \boldsymbol{\Phi})^{-1} \boldsymbol{\Phi}^\top \mathbf{y}$$

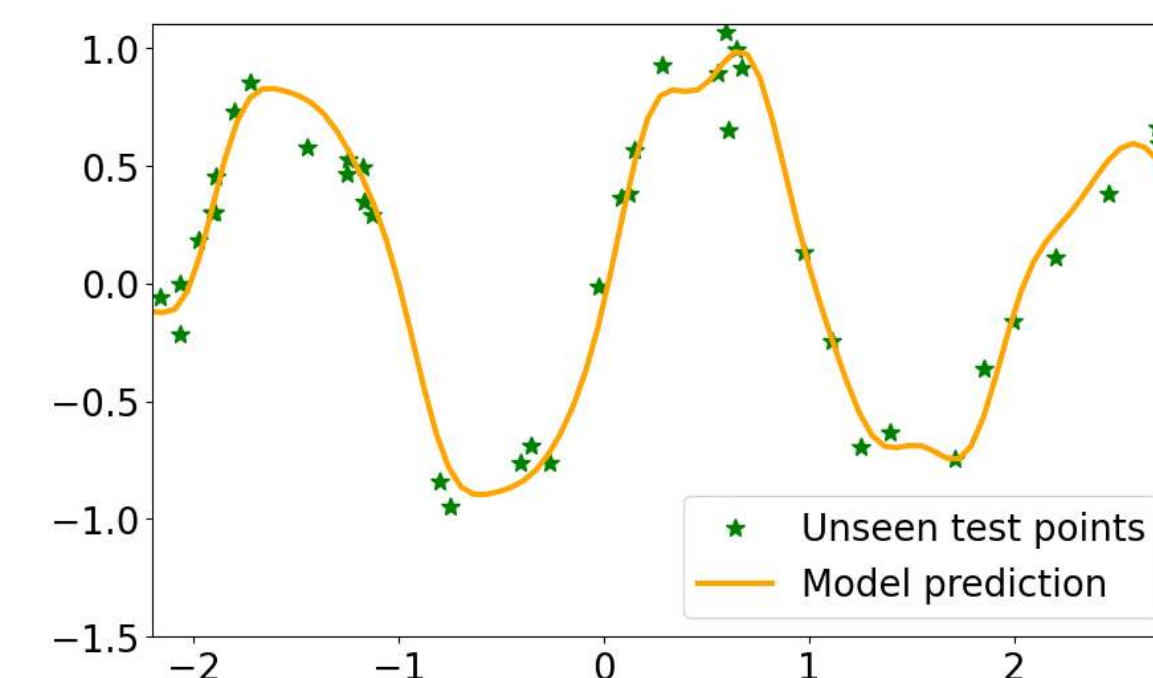
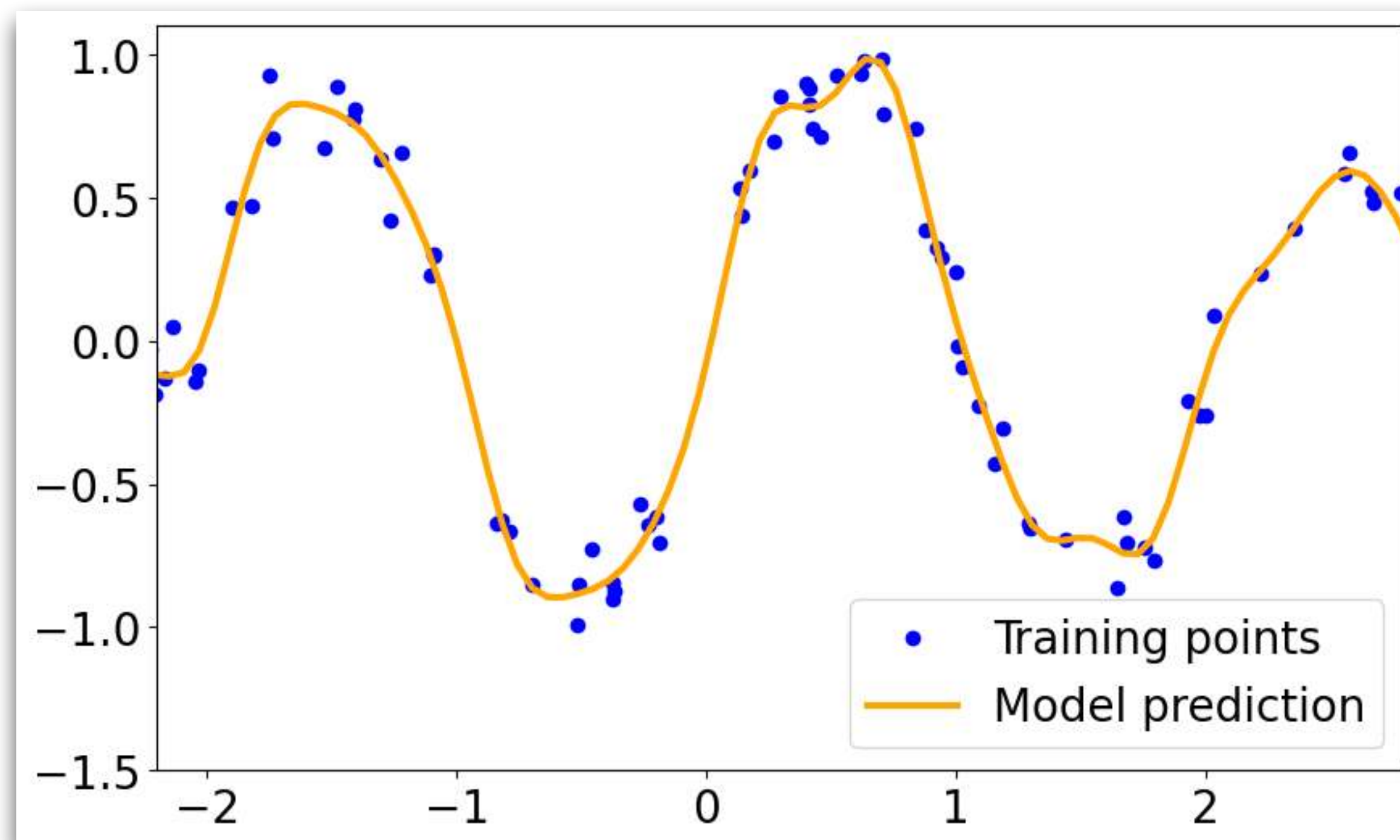
■ 最適解が閉じた形式で求まった

【用語】 線型最小二乗回帰における $\nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}) = \mathbf{0}$ という式を正規方程式（normal equation）という。正規方程式という用語の由来：方程式 $\boldsymbol{\Phi}^\top (\boldsymbol{\Phi} \boldsymbol{\theta} - \mathbf{y}) = \mathbf{0}$ は、 $(\boldsymbol{\Phi} \boldsymbol{\theta} - \mathbf{y})^\top \boldsymbol{\Phi} = \mathbf{0}^\top \Leftrightarrow \langle \boldsymbol{\Phi} \boldsymbol{\theta} - \mathbf{y}, \boldsymbol{\Phi}(\cdot) \rangle = 0$ ，すなわち $(\boldsymbol{\Phi} \boldsymbol{\theta} - \mathbf{y}) \perp \text{range}(\boldsymbol{\Phi})$ と同値。つまり、 $\text{range}(\boldsymbol{\Phi})$ に対する法線ベクトル（normal vector）になっていることを表す方程式なので、上記の名前になっている。閉じた形式（closed-form）は、四則演算や指数・対数、冪乗（べき乗）等の「よく知られた」関数で表される式を指す通称。

線型モデルでの最小二乗法 > 実行例

三角多項式モデル $f_{\theta}(x) = \theta^{\top} \phi(x)$ を最小二乗法で学習した結果

- ここで, $\phi(x) = (1, \sin(x), \cos(x), \sin(2x), \cos(2x), \dots, \sin(15x), \cos(15x))^{\top}$.



- 評価データによる予測性能評価も妥当

過適合 (過学習)

これまでのあらすじ

- 予測器を作りたい



- モデル f_{θ} を選び、損失関数 ℓ を決め、期待損失が小さいパラメーター θ を探す

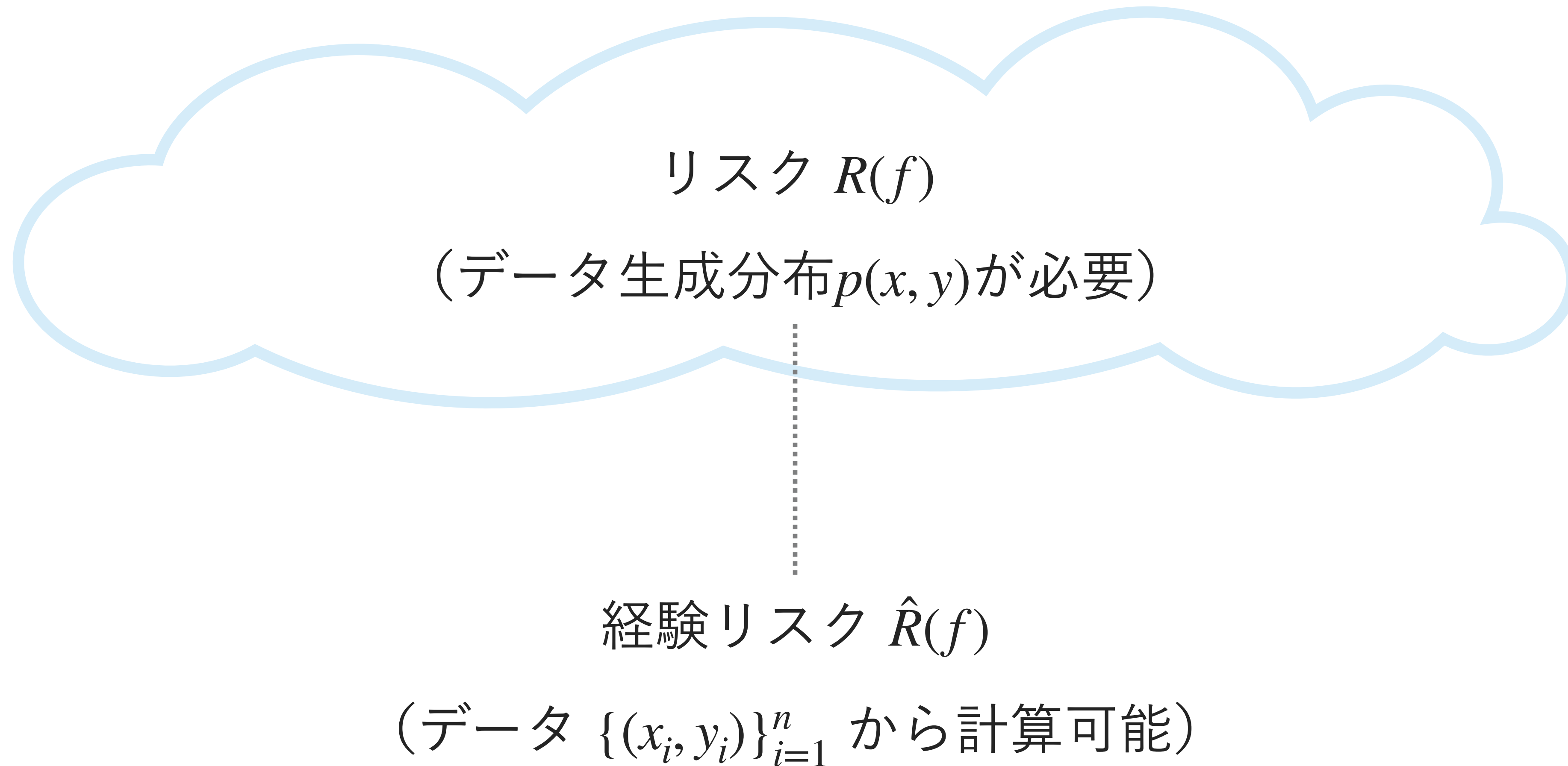
$$R(f_{\theta}) := \mathbb{E} [\ell(Y, f_{\theta}(X))] \quad \leftarrow \quad \text{計算不可能だが小さくしたい}$$

- 現実的には、損失のサンプル平均の最小化なら実行できることを紹介

$$L(\theta) := \hat{R}(f_{\theta}) := \frac{1}{n} \sum_{i=1}^n \ell(y_i, f_{\theta}(x_i)) \rightarrow \text{Minimize}$$

経験リスク

経験リスクは、リスク関数をサンプル近似したもの



経験リスク最小化 (Empirical Risk Minimization)

経験リスクを（何も手を加えずそのまま）目的関数として学習する

- 経験リスクそのものを目的関数として設定して最小化

$$L(\theta) = \hat{R}(f_{\theta}) \rightarrow \text{Minimize } \theta \in \Theta$$

- リスクのサンプル近似を最小化しているので、妥当そうではある
- しかし、愚直にこれだけを実行すると、過適合することがある

経験リスク最小化の限界



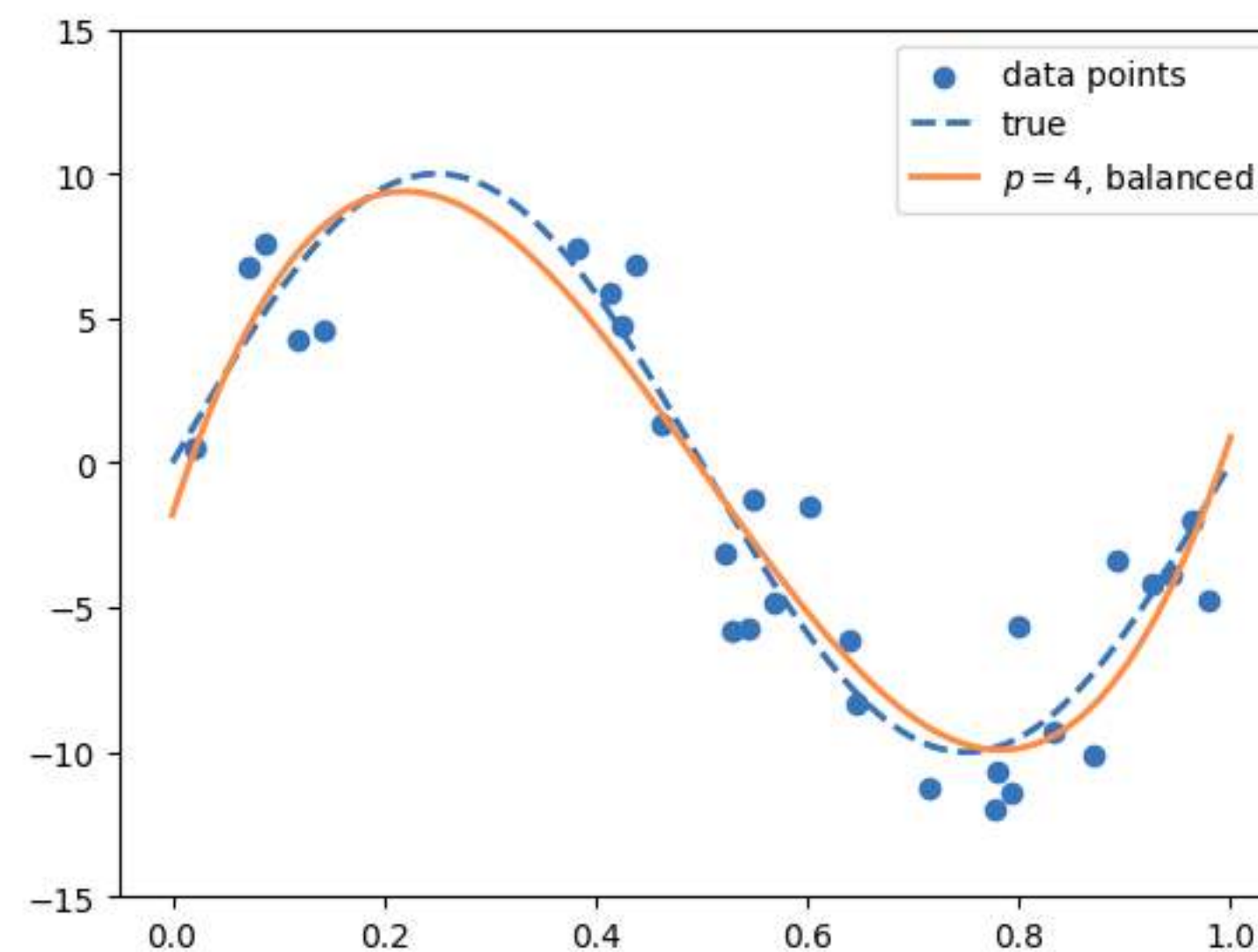
特に複雑なモデルでは、過適合・過学習という問題が起きるようになる

- 経験リスク最小化は、 $R(f_\theta)$ が小さくなることを願って、 $\hat{R}(f_\theta)$ を最小化する手法
 - しかし、狙い通りに $R(f_\theta)$ が小さくなるかどうかは別問題
 - 上手くいくときはうまくいくが、思惑通りにはいかないことの方が多い
- 特に、過適合・過学習という問題が生じる場合があることが知られている

経験リスク最小化

多項式モデルの最小二乗法での学習の成功例

- 4次関数でうまく予測できるような生成分布のデータで実験
 - 4次関数モデルを最小二乗法で学習した結果, よいモデルができた

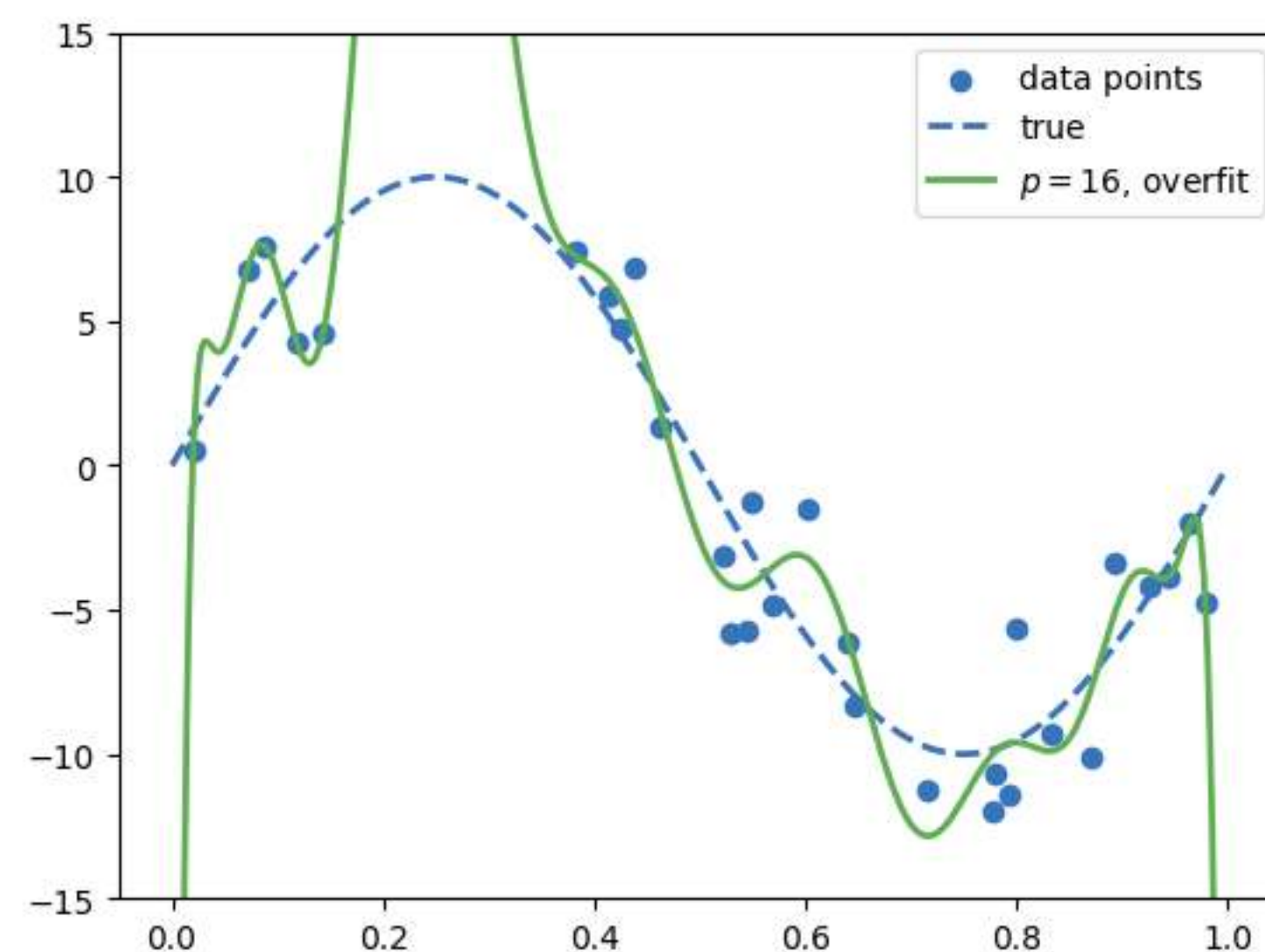


4次関数モデル

過適合（過学習）

パラメーターが多いモデルを単なるERMで学習させると、**過適合**が生じがち

- **過適合（overfit）**：経験リスクは小さくなったが、リスクが小さくない状態
 - つまり、見かけ上の学習はしているが、汎化しないモデルになっている状態



16次関数モデル

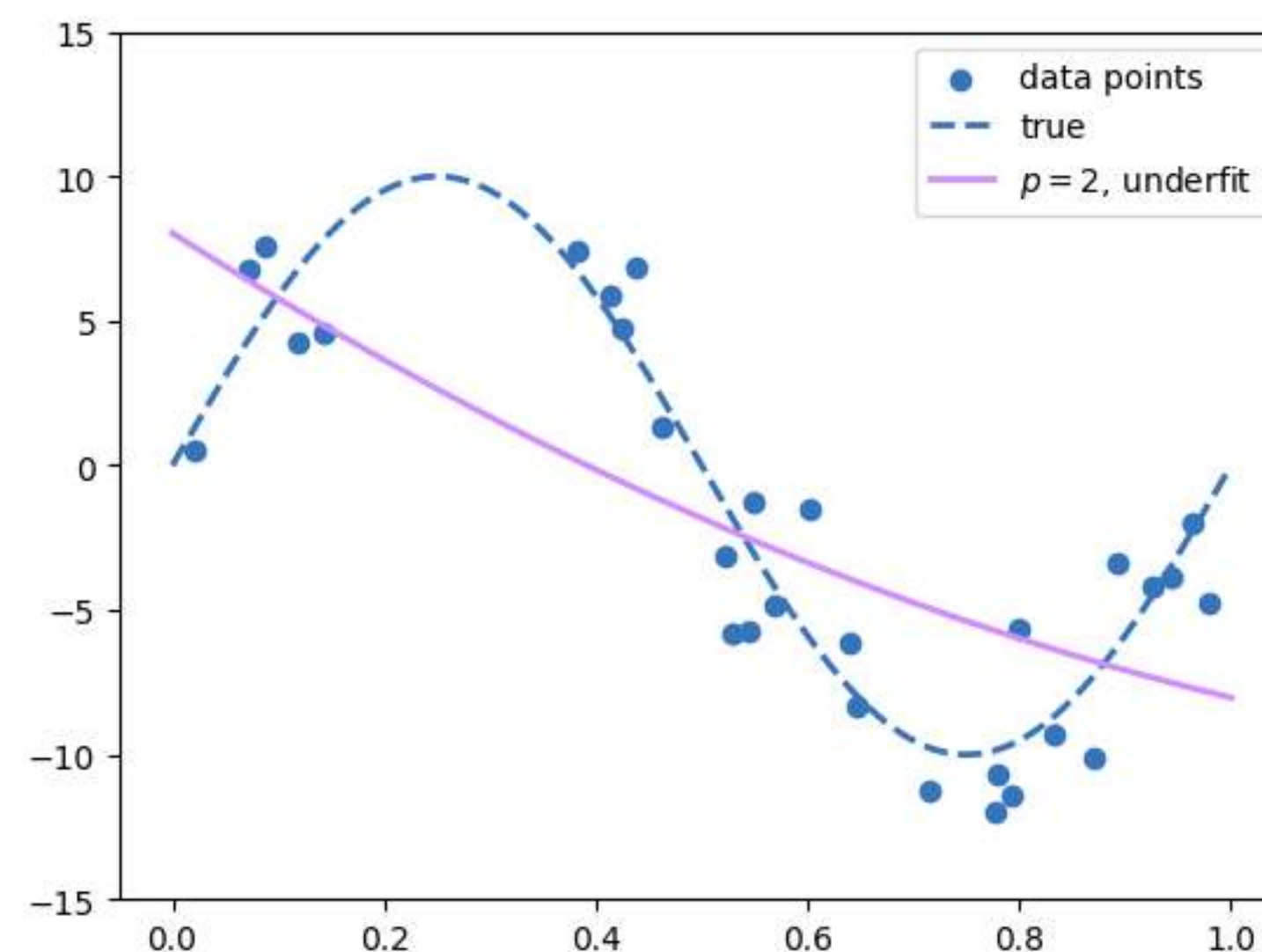
【用語】 過適合という訳のほうが英語とうまく対応するが、日本語では過学習という用語も浸透している。外来語としてオーバーフィットということもある。

【実験ソースコード】 [番号]_overfit_linear_model.ipynb

過少適合

パラメーターが少なすぎるモデルだけでやりくりするのも難しい

- **過少適合 (underfit)** : 最適化後でも経験リスクが大きいままの状態
 - つまり, モデルの柔軟さが足りず, そもそも学習が十分に進まない状態



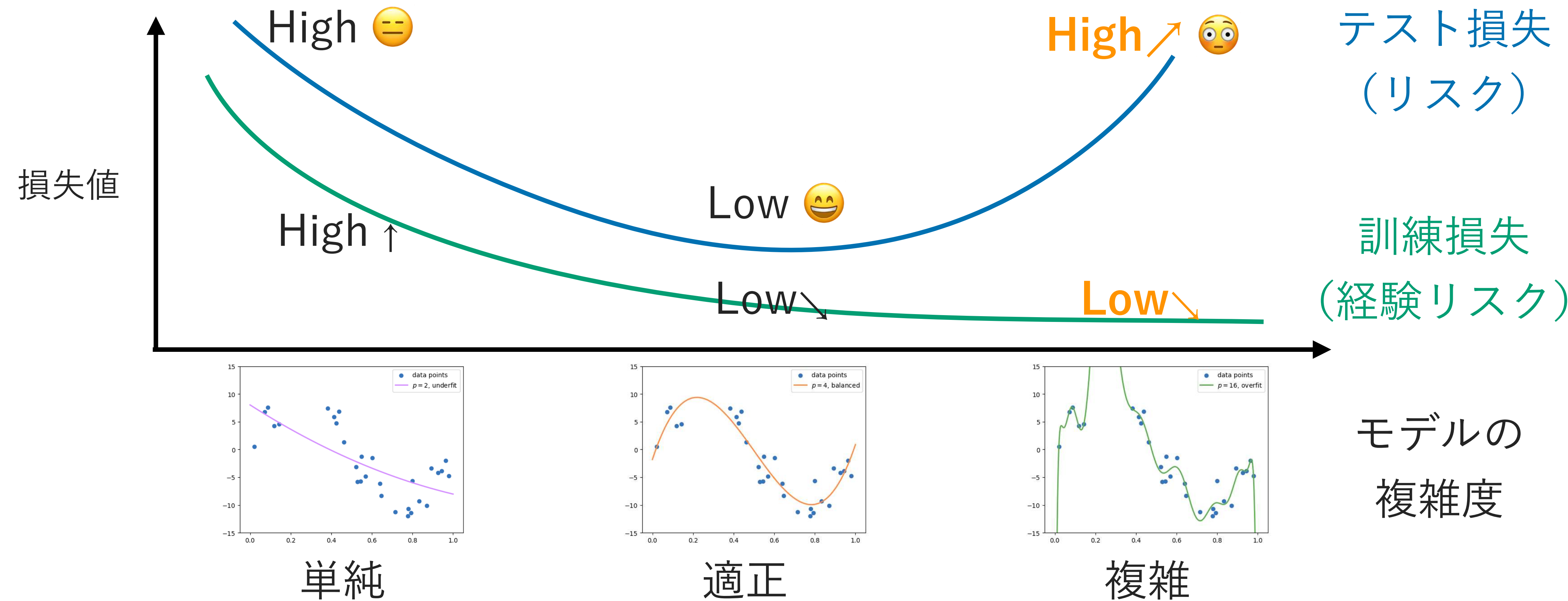
2次関数モデル

【用語】 外来語としてアンダーフィットということも多い.

【実験ソースコード】 [番号]_overfit_linear_model.ipynb

過適合現象（Overfitting）＞ 模式図によるまとめ

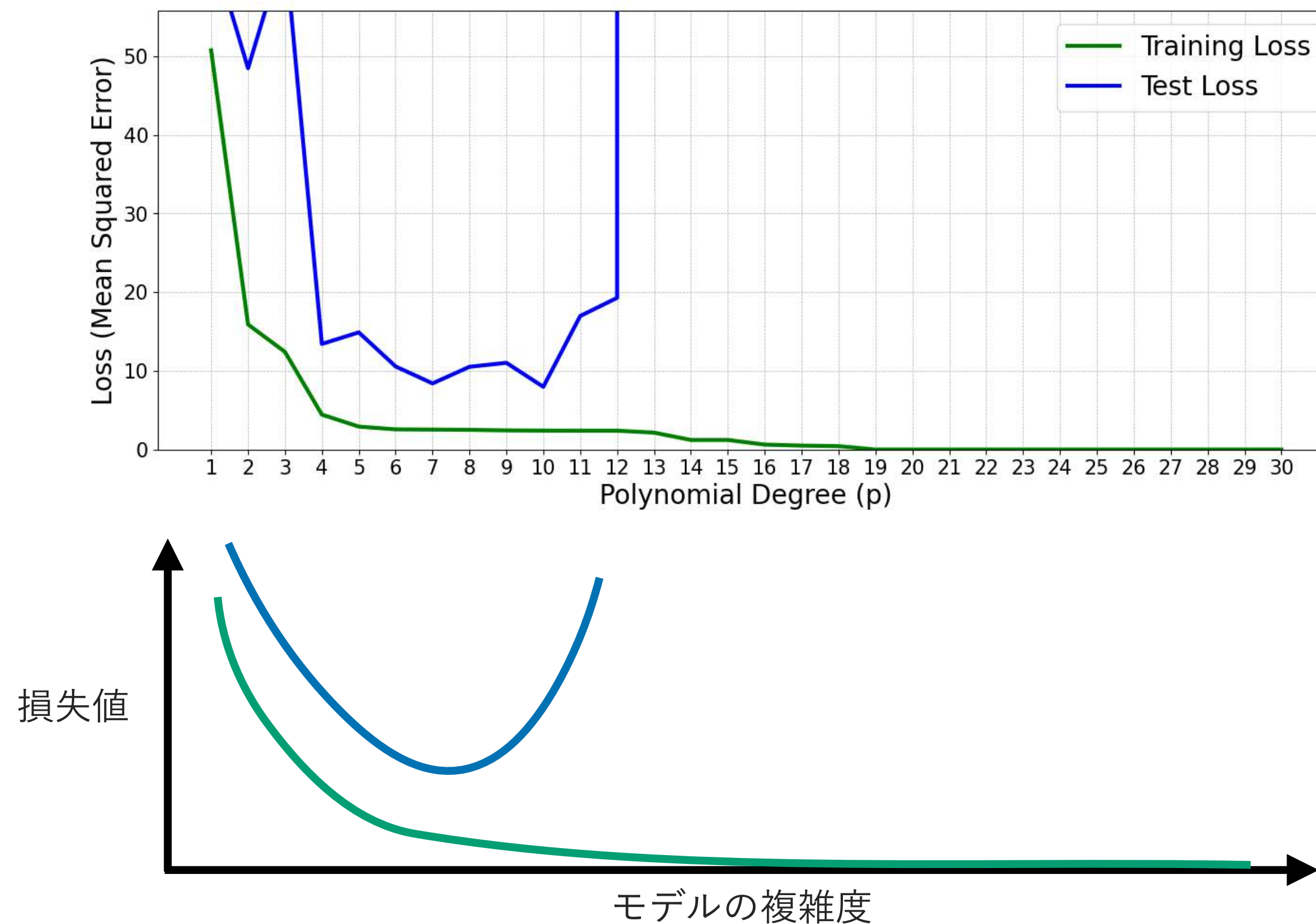
モデルの複雑度を上げていくと，リスクの値が再び上昇する現象



過適合現象（Overfitting） > 学習曲線（Learning Curve）

実験：多項式モデルの最小二乗回帰で次数を上げていった結果

- 横軸：多項式の最大次数，縦軸：損失値として，訓練・評価の損失を描いた図



【実験ソースコード】 [番号]_learning_curve_overfitting.ipynb

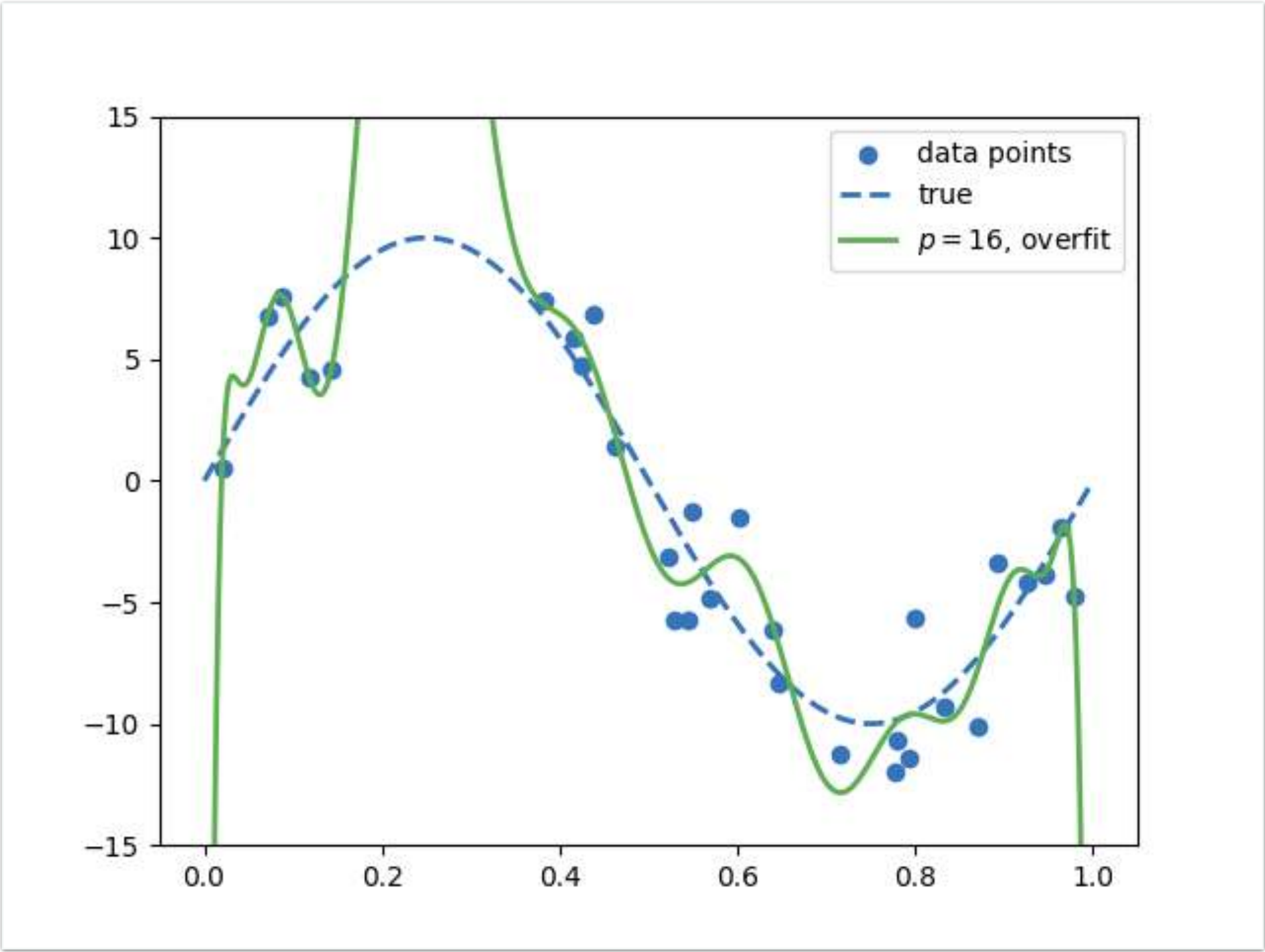
【用語】 横軸が何らかのモデル複雑度，縦軸が何らかの指標値として，訓練と評価の結果をそれぞれプロットした曲線のことを，学習曲線という。

モデルの複雑度と正則化

多項式モデルの過適合とパラメーターの大きさ

上下に激しく変動する関数はパラメーターの絶対値が大きくなって発生

- 入力の微小な差に大きく反応 = パラメーターの絶対値が大きい
- 訓練データ点を通過するように辻褄を合わせていると考えられる



	r=2	r=4	r=16
θ_0	8.0	-1.8	-87.1
θ_1	-23.4	112.3	10466.1
θ_2	7.3	-327.4	-487005.0
θ_3		219.0	12103103.6
θ_4		-1.2	-180067412.9
θ_5			1723030777.0
θ_6			-11159264579.3
θ_7			50782142907.2
θ_8			-166670522119.7
θ_9			400990179401.2
θ_{10}			-712278442253.3
θ_{11}			932198468910.3
θ_{12}			-887330785442.5
θ_{13}			597242347925.3
θ_{14}			-269343089344.4
θ_{15}			73003666266.9
θ_{16}			-8989291594.0

基本知識の確認＞ベクトルの大きさ（ノルム）

ベクトルの大きさ（長さ）はノルムで測る

■ ノルム (norm) $\|x\|$

- 性質： $\|x\| \geq 0$, かつ, $(\|x\| = 0) \Leftrightarrow (x = \mathbf{0})$
- 性質： $\|\alpha x\| = |\alpha| \cdot \|x\|$ （斉次性）, $\|x + y\| \leq \|x\| + \|y\|$ （三角不等式）

■ 内積から誘導されるノルム $\|x\|_{\ell^2}$ （2-ノルム）

$$\|x\|_{\ell^2} = \sqrt{\langle x, x \rangle} = \sqrt{x_1^2 + x_2^2 + \cdots + x_d^2}$$

- 最も有名なノルム（断りがなければこれ）

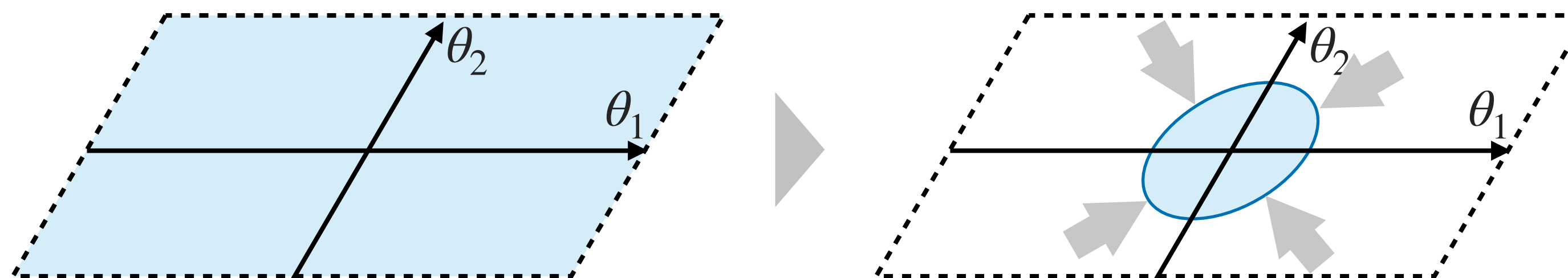
正則化＞L2正則化付き経験リスク最小化

2-ノルムを用いた罰則項を目的関数に追加して，過適合を防止

■ L2正則化（L2 regularization）

$$L(\boldsymbol{\theta}) := \underbrace{\hat{R}(f_{\boldsymbol{\theta}})}_{\text{損失項}} + \underbrace{\lambda \cdot \|\boldsymbol{\theta}\|^2}_{\text{正則化項}} \rightarrow \text{最小化}$$

- $\lambda \geq 0$ は**正則化係数（regularization coefficient）**という（学習時には固定）

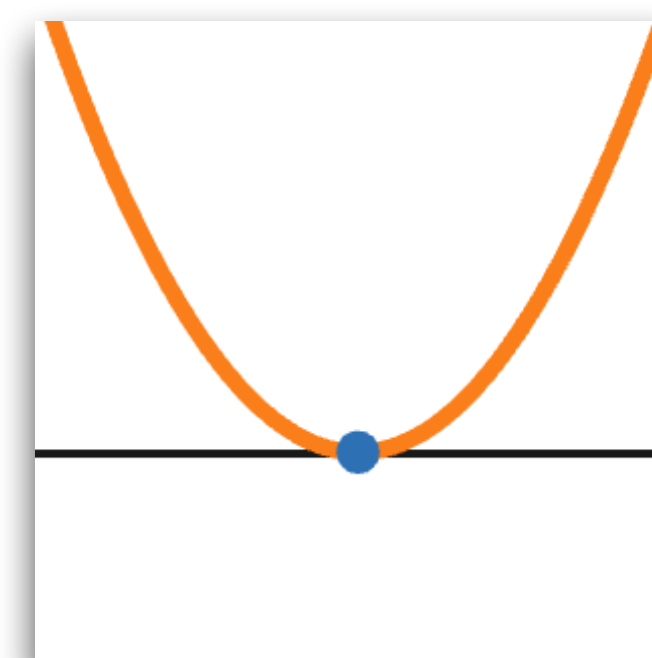


線型モデルでのL2正則化付き最小二乗法の解

一般の線型モデルでのL2正則化付き最小二乗法の解を求めてみよう

- 線型モデル $f_{\theta}(x) = \theta^{\top} \phi(x)$ の場合に、最適化問題

$$\text{Min}_{\theta \in \mathbb{R}^{B+1}} \frac{1}{2} \sum_{i=1}^n (y_i - f_{\theta}(x_i))^2 + \frac{\lambda}{2} \|\theta\|^2$$



を解き、解となるパラメーター $\hat{\theta}$ を求めよ。

- 記号として以下を使う

$$\Phi := \begin{pmatrix} \phi(x_1)^{\top} \\ \vdots \\ \phi(x_n)^{\top} \end{pmatrix} = \begin{pmatrix} \phi_0(x_1) & \cdots & \phi_B(x_1) \\ \vdots & \ddots & \vdots \\ \phi_0(x_n) & \cdots & \phi_B(x_n) \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix}$$

正則化＞線型モデルのL2正則化付き最小二乗法

正則化無しの場合と同じ方法で最適解の公式を得られる

- 正則化付きの目的関数は、 $\lambda\|\boldsymbol{\theta}\|^2 = \lambda\boldsymbol{\theta}^\top\boldsymbol{\theta} = \boldsymbol{\theta}^\top(\lambda\boldsymbol{I})\boldsymbol{\theta}$ に注意すると、

$$L(\boldsymbol{\theta}) = \frac{1}{2}(\mathbf{y} - \boldsymbol{\Phi}\boldsymbol{\theta})^\top(\mathbf{y} - \boldsymbol{\Phi}\boldsymbol{\theta}) + \boldsymbol{\theta}^\top\left(\frac{\lambda}{2}\boldsymbol{I}\right)\boldsymbol{\theta}$$

- 一階の条件

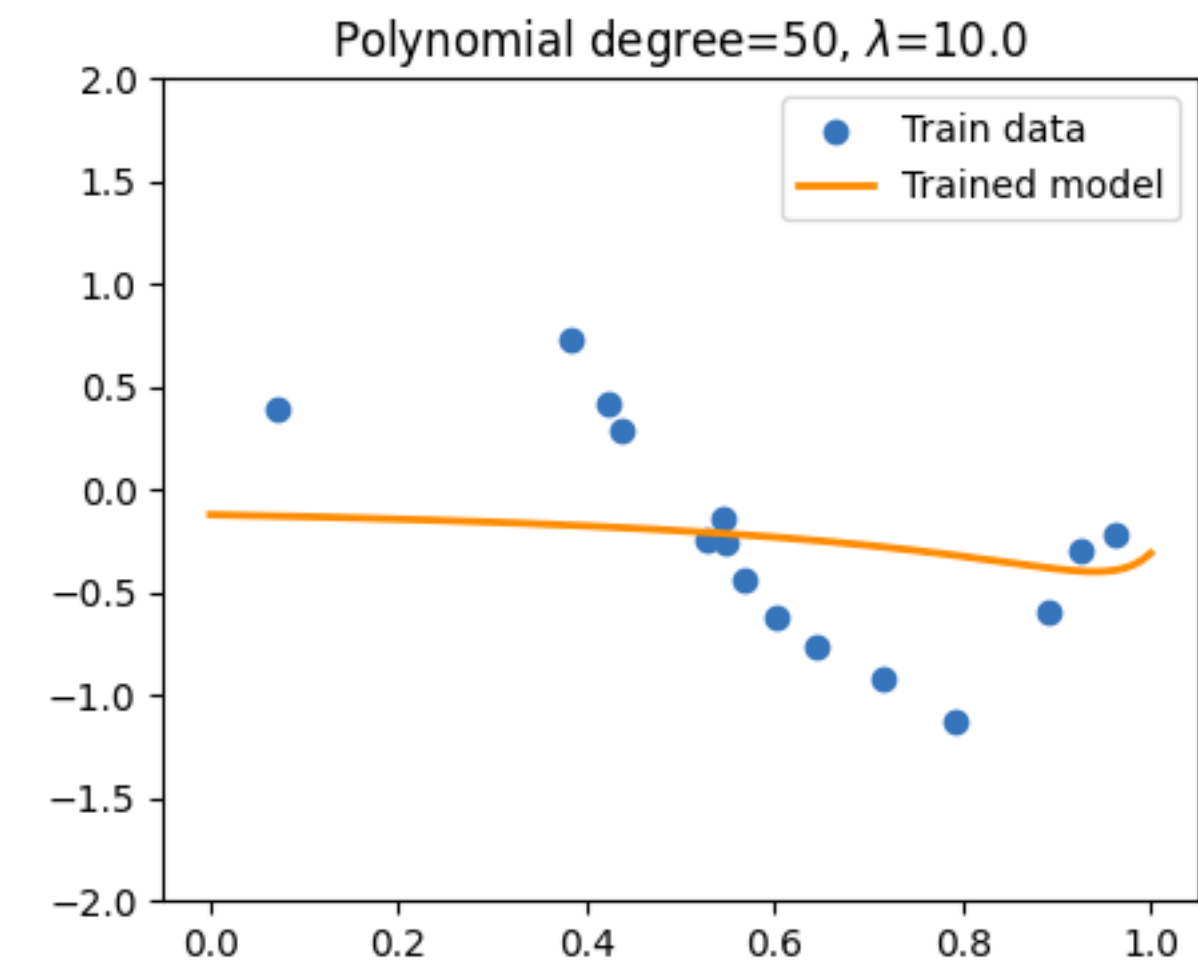
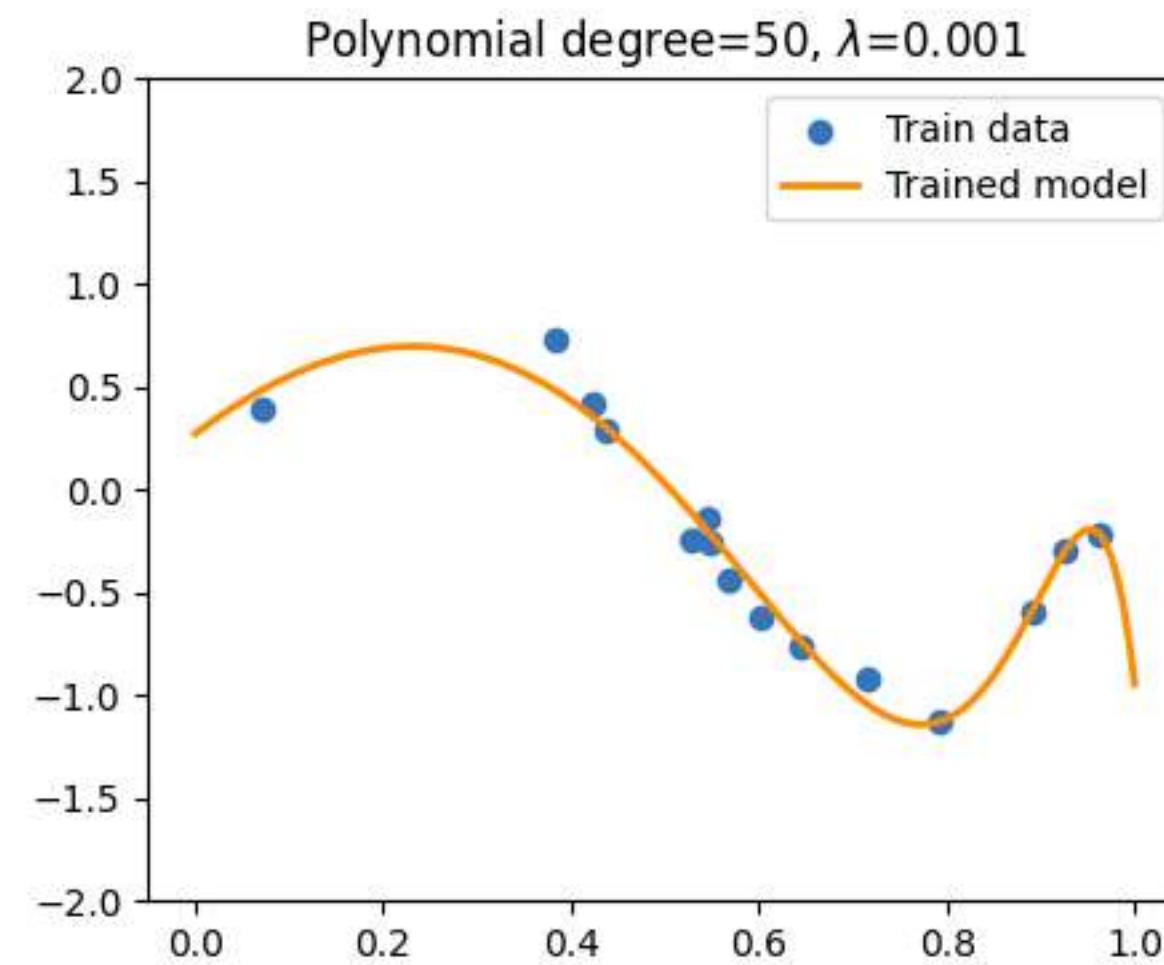
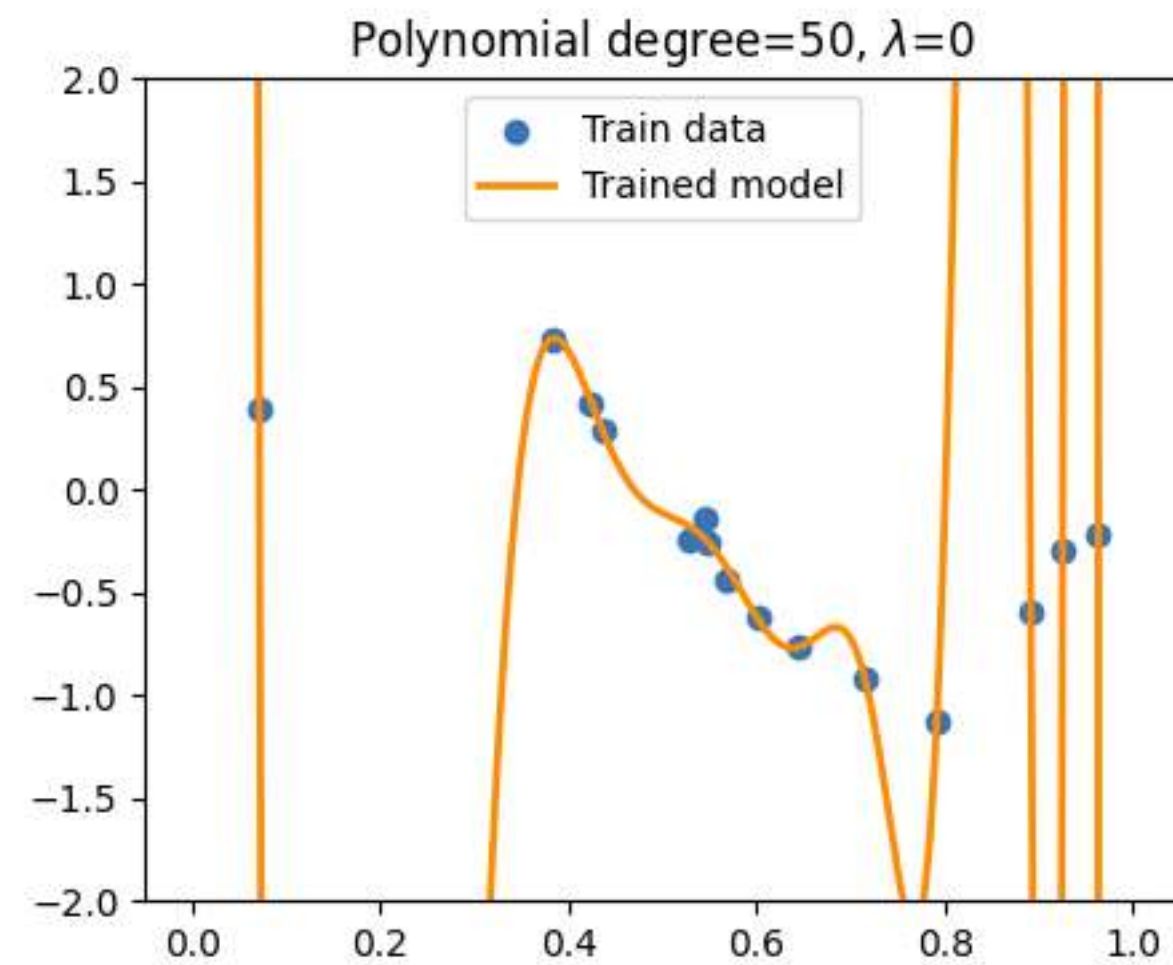
$$\nabla_{\boldsymbol{\theta}}L(\boldsymbol{\theta}) = (\boldsymbol{\Phi}^\top\boldsymbol{\Phi} + \lambda\boldsymbol{I})\boldsymbol{\theta} - \boldsymbol{\Phi}\mathbf{y} = \mathbf{0}$$

- よって、最適解の公式として次を得られる。

$$\hat{\boldsymbol{\theta}} = (\boldsymbol{\Phi}^\top\boldsymbol{\Phi} + \lambda\boldsymbol{I})^{-1}\boldsymbol{\Phi}^\top\mathbf{y}$$

正則化＞線型モデルのL2正則化付き最小二乗法＞実行例

L2正則化の強さを操作することで、過適合をコントロールできる



正則化無し 0 $\xrightarrow{\lambda}$ 正則化強

- 正則化係数 λ の大きさによって、正則化の強さを調整し、過適合を抑制できる

【補足】 正則化が強すぎると $\lambda \rightarrow +\infty$ で $\theta \simeq 0$ が解となり、例えば線型モデルなら定数関数になってしまう。

【実験ソースコード】 [番号]_l2_regularized_linear_regression.ipynb

正則化 (Regularization)

より単純なモデルが学習されるようにする仕掛けのこと

- 意図して組み込む明示的な正則化と、意図せず機能する暗黙的な正則化がある

明示的な正則化 (explicit regularization)

- 最適化問題の目的関数や制約式に手を加えるなど
- 例) L2正則化

暗黙的な正則化 (implicit regularization)

- 選んだ最適化法の傾向など明示的でない全ての正則化
- 例) 確率的勾配法の性質

過適合の防止



過適合を防止するには、モデルを適切に制限する必要がある

- モデルの複雑度を抑えるためには、いくつかの方法がある
 - モデルや学習過程の**正則化**を行う
 - モデルに固有の、複雑度を調整できる方法もある（例：多項式回帰の次数）
- どのぐらいの正則化を施せばよいか

ロードマップ

